

國立中正大學

資訊工程研究所碩士論文

採用 16 奈米製程的無人機即時障礙物
避障 CNN-Vision Transformer 混合硬
體加速器

**A 16 nm Hybrid CNN-Vision
Transformer Hardware Accelerator for
Real-Time UAV Obstacle Avoidance**

研究生：葉絨萱

指導教授：鍾菁哲 博士

中華民國 一一四 年 八 月



國立中正大學碩士學位論文考試審定書

資訊工程學系

研究生 葉絨萱 所提之論文

採用 16 奈米製程的無人機即時障礙物避障 CNN-Vision Transformer
混合硬體加速器 A 16 nm Hybrid CNN-Vision Transformer Hardware
Accelerator for Real-Time UAV Obstacle Avoidance
經本委員會審查，符合碩士學位論文標準。

學位考試委員會
召集人

李順銘 簽章

委員

李順銘

鍾秉哲

成 毅

黃崇力

指導教授

鍾秉哲

簽章

中華民國 114 年 7 月 3 日

摘要

無人機在災害監控、物流運輸與空中偵察等應用中越來越普及，但在複雜環境中自主避障仍是一大挑戰。若無法即時辨識障礙物並做出正確應對，將嚴重影響飛行安全與任務執行效率。因此，發展具備即時感知與高精度判斷能力的模型，以提供後續避開障礙物任務所需的決策資訊，對提升無人機自主性具有關鍵意義。

本研究提出 CFVT 模型。該模型採用混合架構，將基於 CNN 的局部特徵提取與基於 Transformer 的全域依賴建模相結合，從而提升了碰撞預測的準確性與整體效能，最終達到 97.32% 的準確率。為了提高硬體實現效率，Transformer 模組透過硬體友善的多頭自注意力機制進行精簡，並以泰勒級數展開來近似 softmax 運算，在保留準確度的同時顯著降低計算複雜度，實現了 6.891 GFLOPS/W 的效率。此外，透過模型簡化與運算流程優化，CFVT 具備即時推論能力，能有效降低延遲，並確保其適用於資源受限的邊緣裝置。實驗結果顯示，CFVT 可達到 24FPS 的幀率，證實其在即時性與能耗受限的 UAV 應用中具有效性。

關鍵字：無人機、避障系統、CNN、Vision Transformer、低功耗晶片

Abstract

Unmanned aerial vehicles (UAVs) have become increasingly prevalent in disaster monitoring, logistics, and aerial reconnaissance. However, autonomous obstacle avoidance in complex environments remains a major challenge. Failure to promptly identify obstacles and respond appropriately can severely impact flight safety and mission effectiveness. Therefore, developing a model capable of accurate and timely prediction is essential to provide critical information for subsequent obstacle avoidance tasks, thereby enhancing UAV autonomy.

This paper proposes the CNN Fusion Vision Transformer (CFVT) model. The model adopts a hybrid architecture that fuses CNN-based local feature extraction with Transformer-based global dependency modeling, thereby enhancing collision prediction accuracy and overall performance, achieving an accuracy of 97.32%. To improve hardware implementation efficiency, the Transformer module is streamlined with hardware-friendly multi-head self-attention, while the softmax operation is approximated using a Taylor series expansion, preserving accuracy and significantly reducing computational complexity, resulting in an efficiency of 6.891 GFLOPs/W.

Moreover, real-time inference capability is enabled through model simplification and computational pipeline optimization, which reduces latency and ensures suitability for resource-constrained edge devices. Experimental results demonstrate that CFVT achieves a frame rate of 24 FPS, confirming its effectiveness for real-time and energy-constrained UAV applications.

Keywords: UAV, Obstacle Avoidance, CNN, Vision Transformer, Low-Power Chip

Content

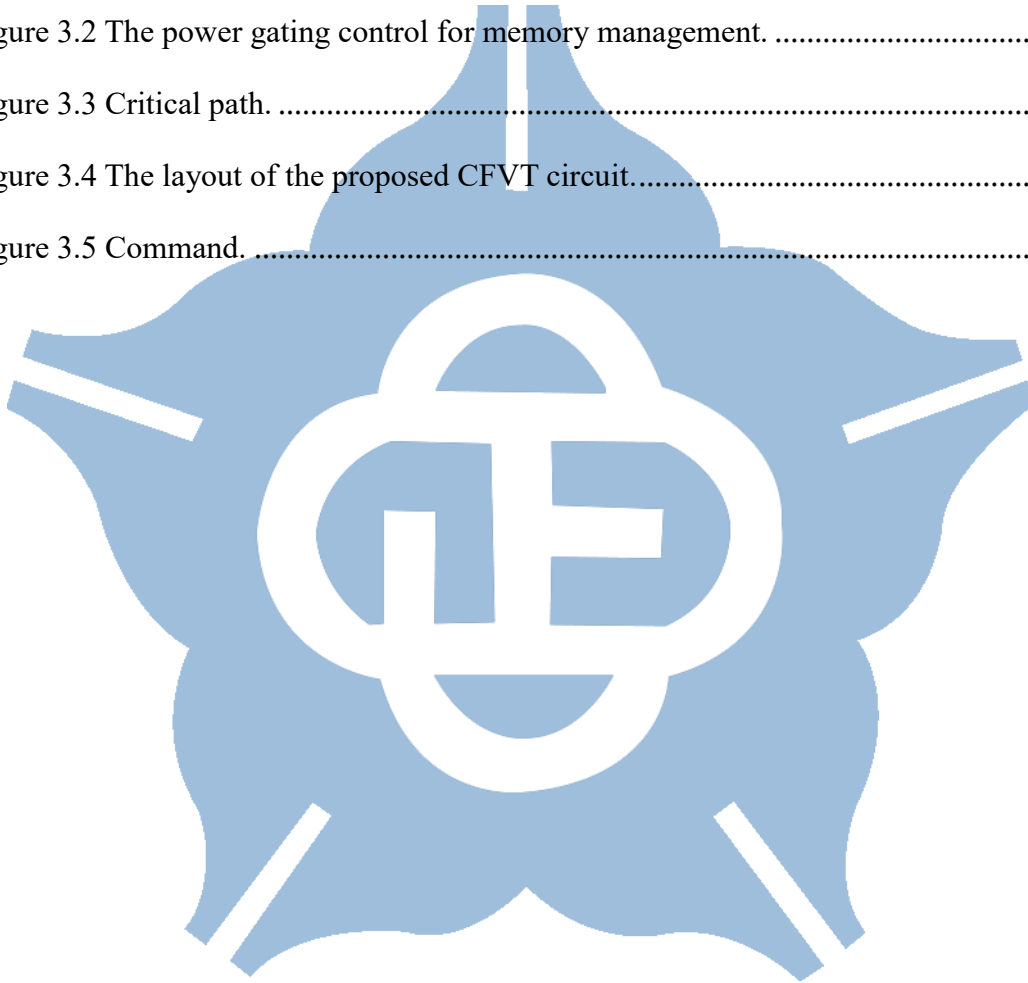
摘要.....	III
Abstract.....	IV
Content.....	V
List of Figures.....	VII
List of Tables.....	IX
Chapter 1 Introduction.....	1
1.1 Introduction to Unmanned Aerial Vehicles and Collision Avoidance	
Dataset.....	1
1.2 Introduction to Convolution Neural Network.....	4
1.3 Introduction to Transformer.....	12
1.4 Lightweight Transformers for Mobile Devices.....	19
1.5 Introduction to related work.....	22
1.6 Hardware-Friendly Softmax Approximations in Transformer	27
1.7 Quantization Methods in Neural Network	32
1.8 Motivation.....	36
1.9 Thesis Organization.....	38
Chapter 2 Software Implementation.....	39
2.1 Architecture Overview	39
2.2 Dataset Preprocessing.....	41
2.3 Evaluation and Selection of Feature Extraction Modules.....	43
2.4 Implementation of CNNFusionViT(CFVT) and Software Analysis.....	46
2.5 Software Result	52

Chapter 3 Hardware Implementation.....	53
3.1 Trade-offs Between Accuracy and Simplicity in Softmax Alternatives	53
3.2 Fixed-point of Activation	56
3.3 Fixed-point of Weight	58
3.4 CFVT Hardware Accelerator Architecture	66
3.5 Power Gating in Memory Management.....	70
3.6 Hardware Implementation Analysis and Comparison	73
3.7 Automatic Place and Route (APR) Implementation Process.....	76
3.8 Summary.....	77
Chapter 4 Conclusion and Future works.....	79
4.1 Conclusion	79
4.2 Future works	81
Reference.....	82

List of Figures

Figure 1.1 Sample images of ColANet dataset [8] .	3
Figure 1.2 The architecture of LeNet-5 CNN used for digit recognition [10] .	7
Figure 1.3 The architecture of AlexNet [11] .	7
Figure 1.4 Residual block [12] .	8
Figure 1.5 Channel shuffling [14] .	9
Figure 1.6 Fire module [15] .	9
Figure 1.7 Depthwise separable convolution [16] .	10
Figure 1.8 The architecture of MobileNetV2 block [17] .	11
Figure 1.9 The architecture of Transformer [18] .	13
Figure 1.10 Multi-Head Self-Attention [18] .	13
Figure 1.11 The architecture of ViT [22] .	16
Figure 1.12 Distillation Process [23] .	17
Figure 1.13 An illustration of the shifted window approach [24] .	18
Figure 1.14 Hierarchical feature maps [24] .	18
Figure 1.15 Overview of Mobile-Former [25] .	20
Figure 1.16 The architecture of MobileViT [26] .	20
Figure 1.17 The architecture of VGG16 [8] .	22
Figure 1.18 The architecture of DCA [27] .	23
Figure 1.19 The MobilenetV3 block [29] .	24
Figure 1.20 BNN to LUTNet architectural transformation for a single channel [30] .	28
Figure 1.21 CORDIC Softmax [31] .	29
Figure 1.22 Hardware architecture of SM-LUT module [32] .	30
Figure 2.1 The flow chart for building a lightweight model.	40
Figure 2.2 The structure of ViT block.	43

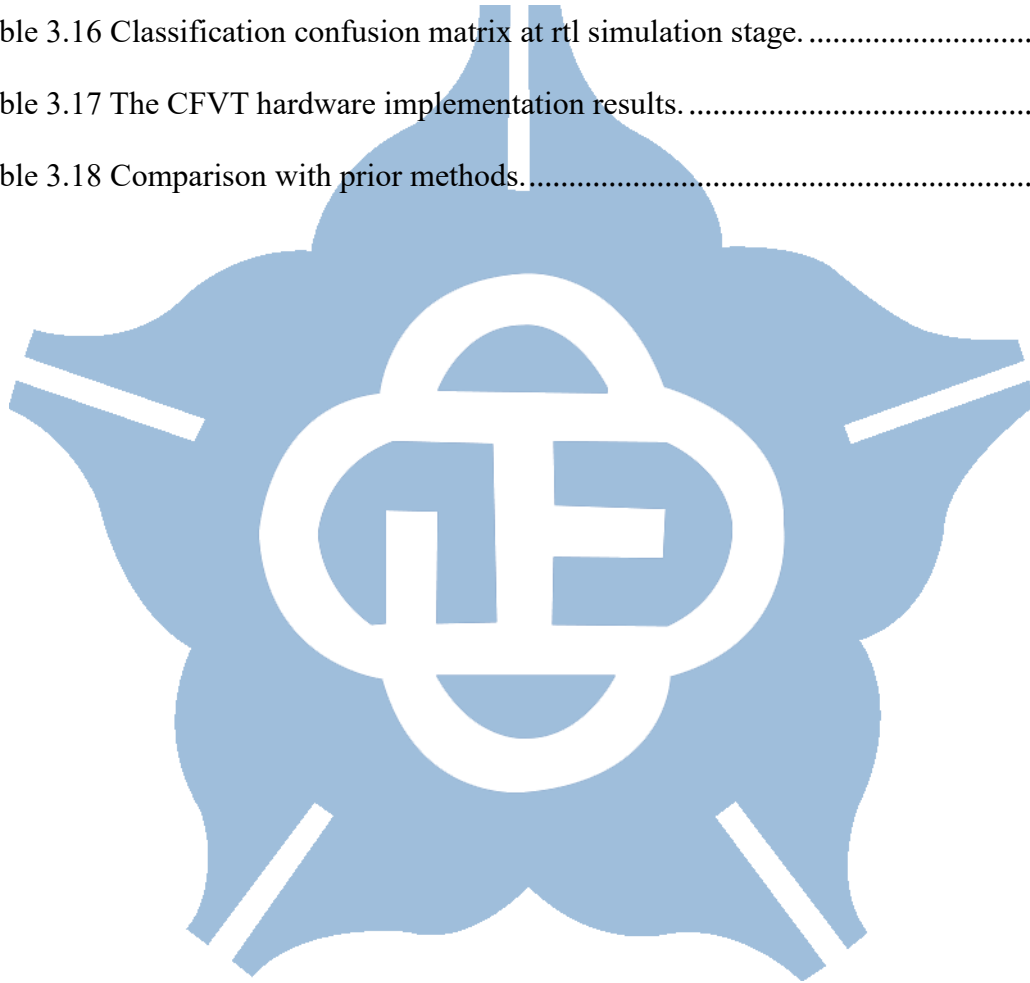
Figure 2.3 The structure of CFVT.....	44
Figure 2.4 The structure of MN1VT. ‘ $\downarrow 2$ ’ indicates stride = 2; the stride is 1 by default.	44
Figure 2.5 The structure of MN2VT. ‘ $\downarrow 2$ ’ indicates stride = 2; the stride is 1 by default.	44
Figure 3.1 The block diagram of proposed CFVT model.....	66
Figure 3.2 The power gating control for memory management.	70
Figure 3.3 Critical path.	74
Figure 3.4 The layout of the proposed CFVT circuit.....	75
Figure 3.5 Command.	76



List of Tables

Table 2.1 The amount of training set and test set in each class	39
Table 2.2 The accuracy of different compression methods across various image sizes.	41
Table 2.3 Performance Comparison of Different Feature Extraction Modules Combined with ViT Block.	45
Table 2.4 The test accuracy of the CFVT model with various layers and channels. ...	46
Table 2.5 The accuracy of different number of ViT block layers.	47
Table 2.6 The test accuracy of the CFV model with various patch size.	48
Table 2.7 The test accuracy of the CFVT model with various token dimension.	48
Table 2.8 The total parameters of CFVT model.	49
Table 2.9 The confusion matrix of CFVT model.	52
Table 2.10 Hyperparameters setting and loss function.	52
Table 3.1 Comparison of standard Softmax and base-2 exponential approximation in terms of accuracy.	54
Table 3.2 The test accuracy of the CFVT model with various taylor polynomial of degree.	54
Table 3.3 The accuracy of Activation quantization with different decimal bits.	56
Table 3.4 The accuracy of weight quantization with different decimal bits in fixed point.	58
Table 3.5 The test accuracy for various decimal bits of BN for γ	60
Table 3.6 The test accuracy for various decimal bits of BN for β	61
Table 3.7 The test accuracy for various decimal bits of LN for γ	62
Table 3.8 The test accuracy for various decimal bits of LN for β	62
Table 3.9 The accuracy of Mean quantization with different decimal bits.	63

Table 3.10 The accuracy of Variance quantization with different decimal bits.	64
Table 3.11 The selection of fixed-point for hardware implementation.	64
Table 3.12 The memory usage for each layer.	68
Table 3.13 The power state in each layer, with a focus on the RAM.	70
Table 3.14 The power state in each layer, with a focus on the ROM.	71
Table 3.15 The test accuracy with different operations.	73
Table 3.16 Classification confusion matrix at rtl simulation stage.	74
Table 3.17 The CFVT hardware implementation results.	75
Table 3.18 Comparison with prior methods.	77



Chapter 1 Introduction

1.1 Introduction to Unmanned Aerial Vehicles and Collision Avoidance Dataset

Unmanned Aerial Vehicles (UAVs), also known as drones, are aerial platforms that perform flight missions through remote control or autonomous navigation. With rapid advancements in sensor technology, wireless communication, artificial intelligence (AI), and energy management, UAVs have been widely adopted across various fields. UAVs not only support the digital transformation of traditional industries but also enable numerous emerging applications. Current major use cases include agricultural monitoring and precision farming [1], disaster response and public safety [2], as well as defense and military operations [3].

Regardless of the specific task, collision avoidance [4] remains a core factor for ensuring mission success and flight safety. In complex and dynamic environments—such as narrow gaps between urban buildings, inspection paths filled with power pylons, or rescue scenes scattered with debris—UAVs must possess reliable collision avoidance capabilities. Failing to do so may lead to equipment damage, data loss, or even pose risks to people and surrounding structures.

To reduce collision risks, both academia and industry have proposed various UAV collision avoidance methods, which can be categorized as follows: Sensor-based methods [5], such as ultrasonic sensors, LiDAR, and optical flow techniques. These rely on real-time distance measurement or image variation analysis to avoid obstacles. These approaches are fast and cost-effective but can be easily affected by environmental factors such as strong light, rain, or fog, which reduce detection reliability.

Rule-based methods [6], including approaches like Artificial Potential Field (APF)

and Velocity Obstacle (VO). These rely on predefined rules for collision avoidance and feature simple algorithms with low computational demands, making them suitable for resource-limited UAV platforms. However, such methods often lack flexibility and may fail in dynamic or unknown environments—for instance, the system can become trapped in local minima.

Simultaneous Localization and Mapping methods [7], which use SLAM to build maps while navigating unknown environments. Combined with global path planning, these methods offer high-accuracy navigation and obstacle avoidance. However, SLAM require significant computational resources, affecting real-time performance and battery life.

Among all approaches, deep learning-based vision methods have emerged as a popular research direction [8]. By employing models like Convolutional Neural Networks (CNNs) or YOLO, UAVs can detect obstacles directly from images and even estimate scene depth. These methods provide strong perception and adaptability but demand large training datasets and high computational resources during inference, which places a heavy burden on edge platforms used in lightweight UAVs.

This study uses ColANet [8] as the reference dataset. ColANet is an open-source dataset specifically designed for UAV collision avoidance research, aiming to improve UAV safety and performance in dynamic environments. As detailed in Section 1.5, ColANet includes diverse conditions such as sunny, cloudy, and nighttime scenes, and contains 3,120 images with collisions and 15,752 images of normal flight, as shown in Figure 1.1.

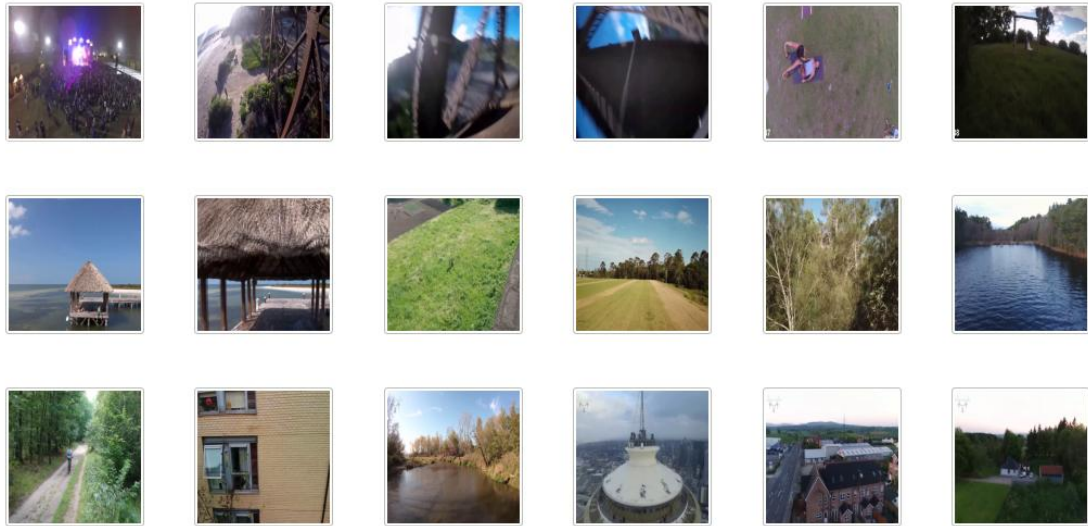
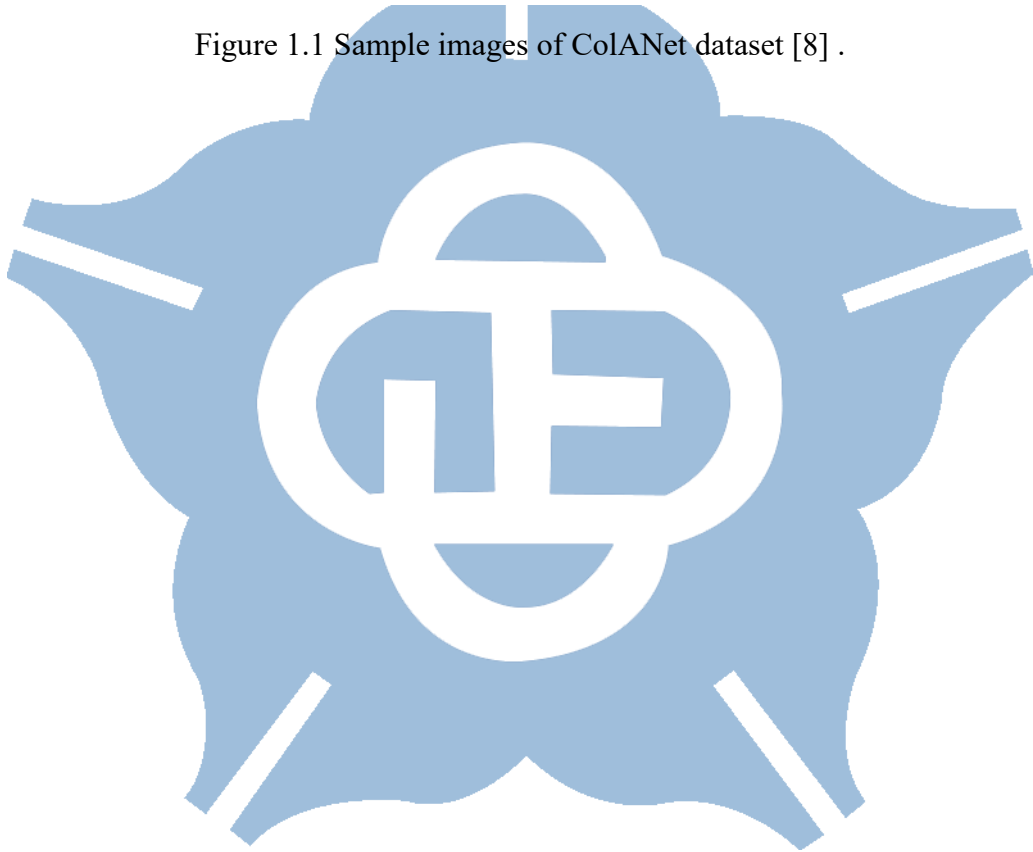


Figure 1.1 Sample images of ColANet dataset [8] .



1.2 Introduction to Convolution Neural Network

Convolutional Neural Networks (CNNs) are deep learning models well-suited for image processing and computer vision tasks. Inspired by the human visual system, CNNs can automatically learn features from images and use them for tasks such as classification, detection, or recognition. A standard CNN typically consists of an input layer, convolutional layers, activation layers, pooling layers, fully connected layers, and an output layer.

Before performing convolution operations, input images are usually resized to a fixed size to ensure consistent network structure and improve training efficiency. Common image resizing methods include nearest neighbor interpolation, bilinear interpolation, and bicubic interpolation. Nearest neighbor interpolation simply selects the closest pixel value, which is computationally efficient but may result in pixelated images. Bilinear interpolation considers the average of four surrounding pixels for smoother results. Bicubic interpolation, which uses a weighted average of 16 surrounding pixels, offers the best visual quality but requires more computation.

The core component of a CNN is the convolutional layer, which performs convolution operations. In this step, small filters slide over the input image and perform multiplication and addition operations. This allows the model to learn features such as edges, textures, and higher-level patterns. After the convolution, a set of feature maps is generated. Each filter produces a unique feature map that reflects specific patterns learned by the model. Together, these maps provide a multi-level representation of the input data, helping the model accurately identify complex visual content.

After the convolution operation, the output passes through an activation layer to introduce non-linearity. Without non-linear transformation, neural networks would be

limited to solving only linear problems, reducing their ability to learn complex patterns. The activation function is applied to the output of each neuron to determine whether it should be activated.

Common activation functions include ReLU, ReLU6, Sigmoid, and Tanh. ReLU6 is a variant of ReLU that clips the output at a maximum value of 6, effectively limiting the neuron's activation range. The constraint helps improve training stability. The mathematical expressions for these activation functions are as follows:

$$ReLU(x) = \max(0, x) \quad (1.1)$$

$$ReLU6(x) = \min(\max(0, x), 6) \quad (1.2)$$

$$Sigmoid(x) = \frac{1}{1+e^{-x}} \quad (1.3)$$

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.4)$$

The pooling layer is used to reduce the spatial dimensions of the image by downsampling, while preserving important information. This not only decreases computational cost but also improves the model's tolerance to spatial variations in the input. Common pooling methods include max pooling and average pooling. Max pooling selects the highest value among neighboring pixels, retaining the most representative feature. In contrast, average pooling computes the average of the surrounding pixels, resulting in a smoother feature map.

In a typical CNN architecture, after passing through several convolutional and pooling layers, the network proceeds to the fully connected layer. The preceding layers progressively extract hierarchical features from the input data, such as edges and textures. Before entering the fully connected layer, the feature maps are flattened into a

one-dimensional vector, which is then processed through weighted summation to integrate the learned features. The final output layer generates the classification or prediction results.

Beyond the basic structure, additional techniques are often incorporated to enhance model performance. For example, Batch Normalization, introduced in [9], normalizes the input of each layer using its mean and standard deviation, and applies trainable scaling and shifting parameters. This technique reduces internal covariate shift, which in turn accelerates training and improves model stability. The mathematical formulation is as follows:

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad (1.5)$$

$$\alpha_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (1.6)$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\alpha_B^2 + \epsilon}} \quad (1.7)$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv BN_{\gamma, \beta}(x_i) \quad (1.8)$$

In 1998, Yann LeCun and his team proposed the LeNet-5 architecture [10], which was successfully applied to handwritten digit recognition. The architecture consists of two convolutional layers, pooling layers, fully connected layers, and Gaussian connection layers. Although LeNet-5 is relatively simple compared to modern architectures, it laid the groundwork for the application of deep learning in image recognition, as illustrated in Figure 1.2.

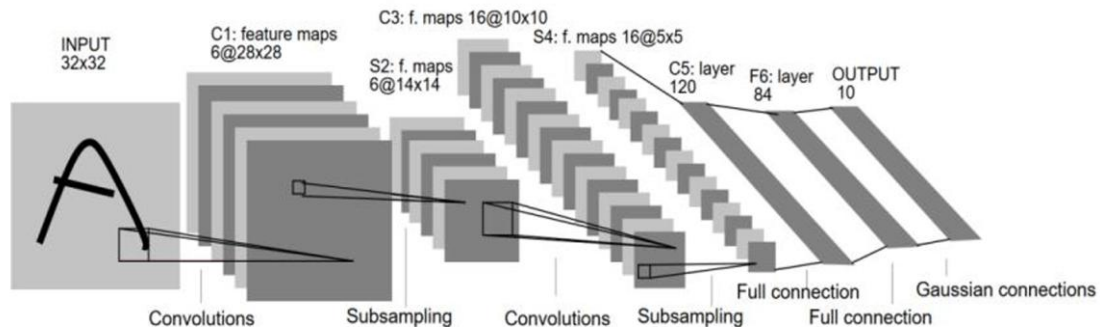


Figure 1.2 The architecture of LeNet-5 CNN used for digit recognition [10] .

In 2012, Alex Krizhevsky and his team proposed AlexNet [11] , which achieved first place in the ImageNet competition and marked a major breakthrough in deep learning for computer vision. AlexNet was the first model to use the ReLU activation function and dropout regularization, significantly improving performance and contributing to the widespread adoption of ReLU in neural network architectures. The network structure is illustrated in Figure 1.3.

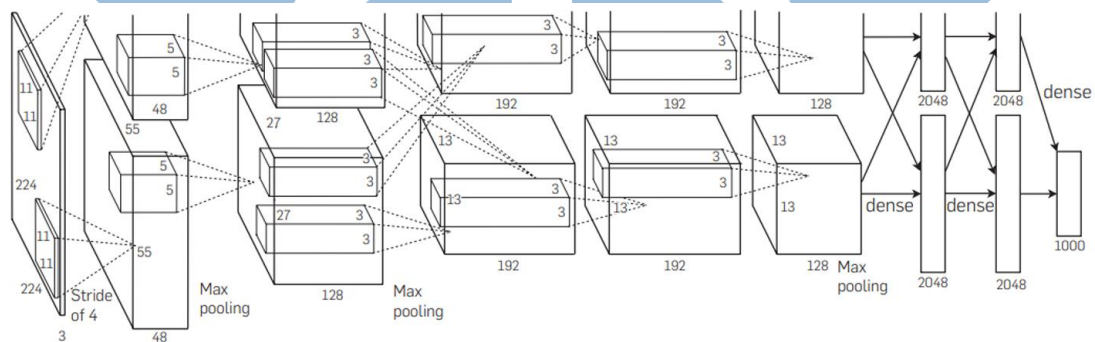


Figure 1.3 The architecture of AlexNet [11] .

To address the vanishing gradient problem in training deep neural networks, K. He and colleagues proposed ResNet [12] . By introducing residual blocks, ResNet allows for the stable training of very deep architectures by enabling the network to learn residual mappings instead of direct transformations. The structure of ResNet is illustrated in Figure 1.4.

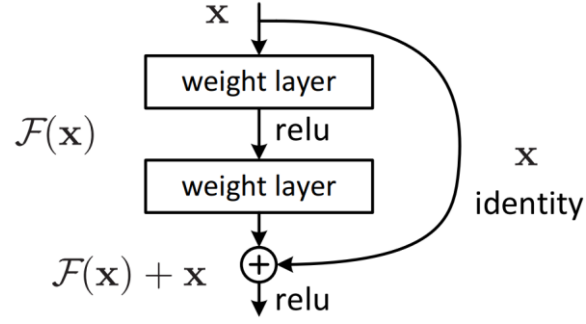


Figure 1.4 Residual block [12] .

In 2014, K. Simonyan and colleagues introduced VGGNet [13] , which featured a deeper architecture and standardized the convolutional kernel size to 3×3 , replacing the larger 5×5 kernels used in AlexNet. VGGNet was released in multiple variants—VGG11, VGG13, VGG16, and VGG19—with VGG16 being the most widely adopted due to its balance of depth and computational efficiency.

Although models such as AlexNet, ResNet, and VGG deliver strong performance, their high computational demands, architectural complexity, and large number of parameters make them unsuitable for deployment on edge devices with limited resources. These challenges have driven the development of lightweight CNNs.

The goal of lightweight CNNs is to achieve efficient computation and low memory usage, making them suitable for applications on edge devices, smartphones, and Internet of Things (IoT) platforms. These models aim to strike a balance between accuracy and computational cost. Representative lightweight architectures include ShuffleNet [14] , SqueezeNet [15] , and MobileNet [16] .

ShuffleNet, developed by the Face++ team, is designed specifically for low-power edge devices. Its key feature is the channel shuffling technique (Figure 1.5), which effectively improves information flow across feature channels.

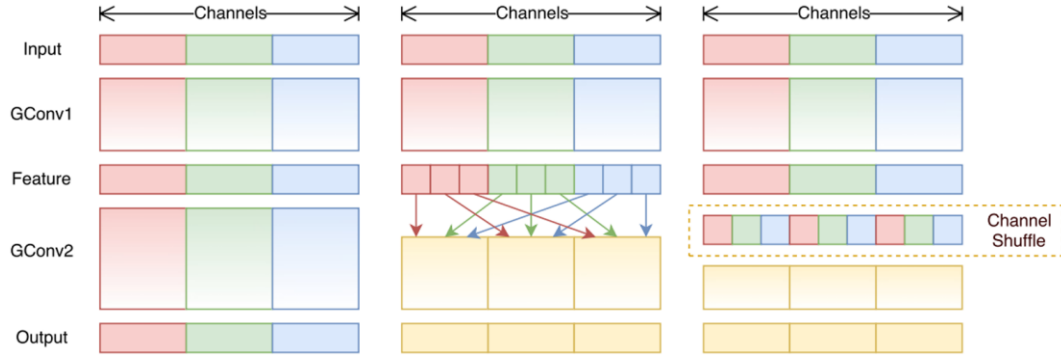


Figure 1.5 Channel shuffling [14] .

SqueezeNet, developed jointly by DeepScale, UC Berkeley, and Stanford University, significantly reduces model parameters while maintaining performance comparable to AlexNet through its unique Fire module design. Each Fire module consists of two components: the squeeze layer, which uses 1×1 convolutions to reduce the number of channels, and the expand layer, which combines 1×1 and 3×3 convolutions to increase the dimensionality of the feature maps, as illustrated in Figure 1.6.

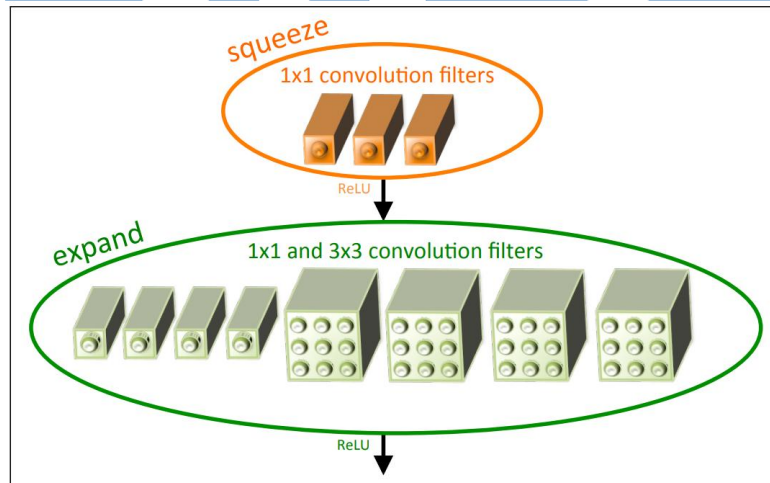


Figure 1.6 Fire module [15] .

MobileNetV1 [16], developed by Google, employs a technique known as depthwise separable convolution to dramatically reduce the number of parameters and computational cost. This method decomposes standard convolution into two steps: first,

a depthwise convolution is applied to each input channel individually; then, a pointwise convolution using 1×1 kernels is used to combine the outputs across channels. The structure is shown in Figure 1.7.

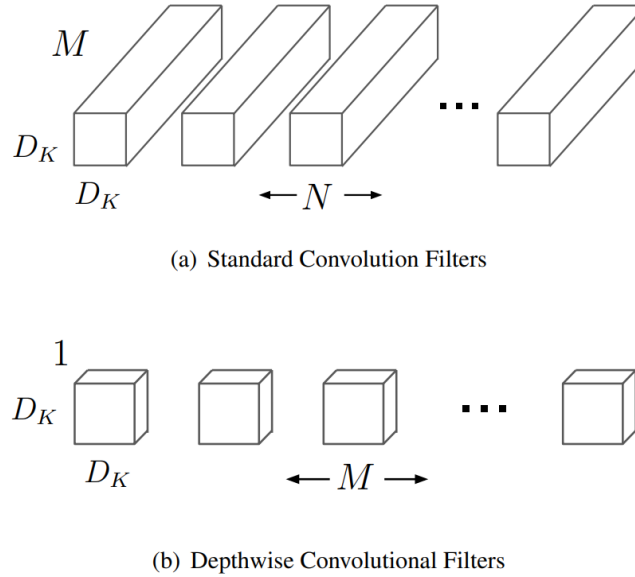


Figure 1.7 Depthwise separable convolution [16] .

MobileNetV2 [17] , developed by Google, is a lightweight convolutional neural network designed for mobile and edge devices. Building on the original MobileNetV1, it introduces two key innovations: the inverted residual structure and linear bottleneck, which enhance model performance while further reducing computational cost. MobileNetV2 continues to utilize depthwise separable convolutions to minimize the number of parameters and computation.

When the stride is set to 1, the spatial resolution of the feature map remains unchanged. In such cases, a residual connection is typically added, allowing the input to be directly summed with the output. This helps preserve fine-grained features and improves information flow. In contrast, when the stride is set to 2, the operation reduces the spatial resolution of the feature map to decrease computation and expand the receptive field. Due to the change in dimensions, residual connections are generally not

applied in these cases to avoid mismatched shapes. The structure is shown in Figure 1.8.

By adopting such a structure, MobileNetV2 achieves a strong balance between accuracy and efficiency, making it suitable not only for performance-sensitive applications but also for deployment on edge devices with limited computing resources.

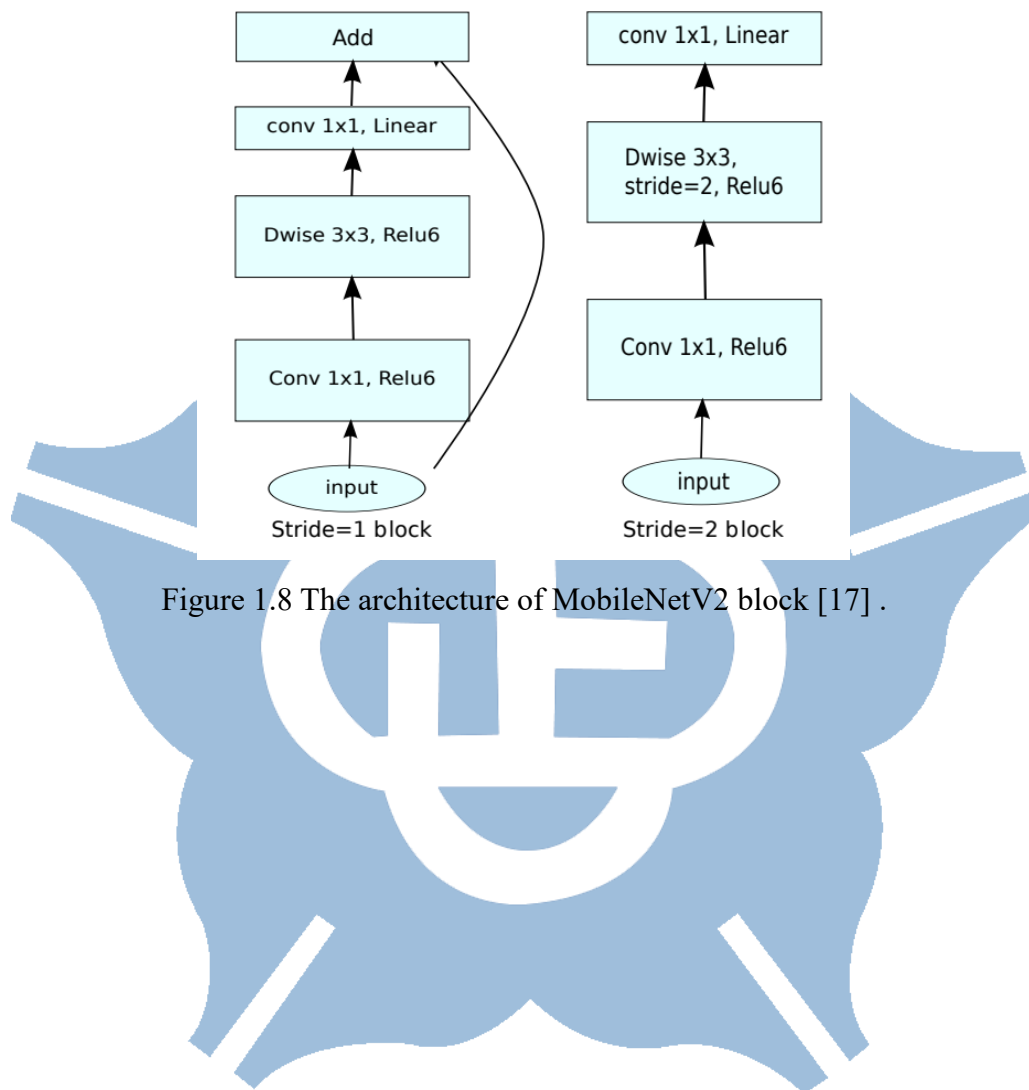


Figure 1.8 The architecture of MobileNetV2 block [17] .

1.3 Introduction to Transformer

Transformer [18] is a deep learning model introduced by the Google team in 2017, originally developed for natural language processing. Unlike traditional recurrent neural networks (RNNs), the Transformer removes the use of recurrence and instead uses an attention mechanism to handle relationships in sequence data. This allows the model to look at all positions in a sequence at the same time, without relying on step-by-step processing. Such an architecture solves the problem of long-distance dependencies and improves training speed thanks to its fully parallel computation.

The Transformer has two main parts: an encoder and a decoder, as shown in Figure 1.9. The encoder turns the input into meaningful representations, and the decoder uses these to produce the output step by step. Each encoder layer includes a multi-head self-attention mechanism and a feedforward neural network. Multi-head attention lets the model learn different features from the data at the same time and better understand long-distance relationships (Figure 1.10). The feedforward network then further processes and strengthens these features, as shown in Equation 1.9.

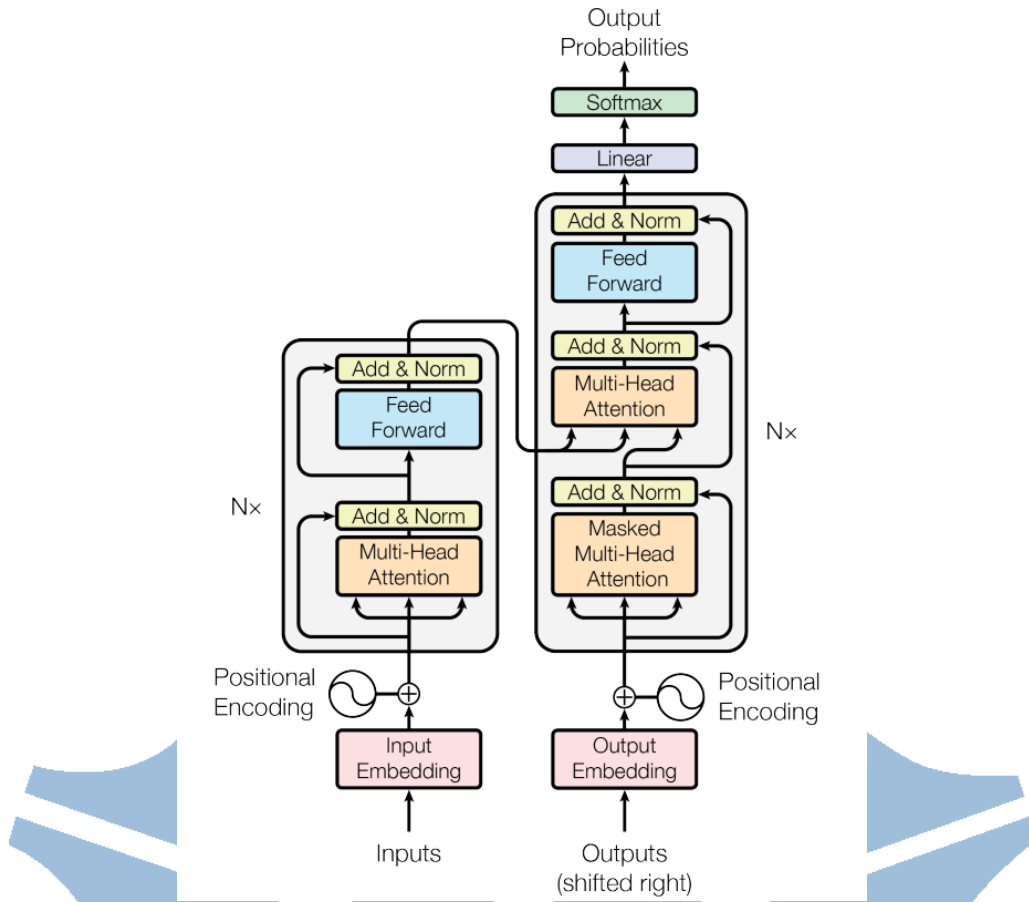


Figure 1.9 The architecture of Transformer [18] .

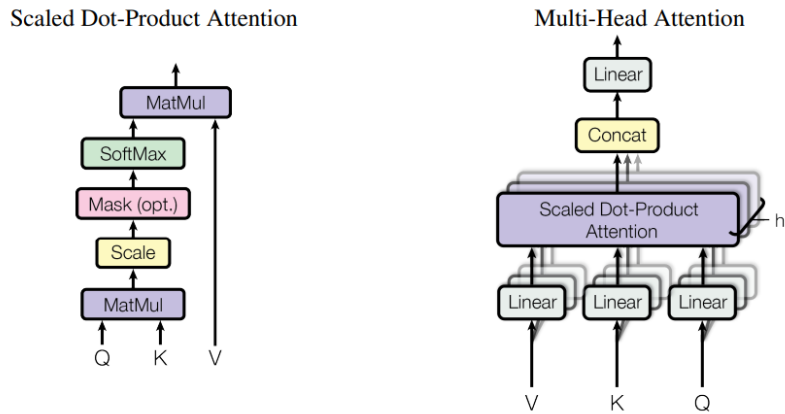


Figure 1.10 Multi-Head Self-Attention [18] .

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (1.9)$$

In the multi-head self-attention mechanism, the Softmax function plays a key role. When computing attention scores, the model first uses matrix operations involving the

query, key, and value to produce initial similarity scores. These scores are then passed through the Softmax function to convert them into a probability distribution, which allows the model to assign reasonable weights to different positions, as shown in Equation 1.10. Specifically, Softmax increases the impact of important positions on the current output while reducing the influence of irrelevant ones, helping the model focus on key information. Although the mathematical form of Softmax is simple, its computation involves exponentiation and floating-point division, which pose challenges for hardware implementation. These issues are discussed in more detail in section 1.6.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.10)$$

To address the Transformer's inability to capture positional information, positional encoding is introduced. This method uses sine and cosine functions at different frequencies to encode each position, allowing the model to understand the relative or absolute positions of elements in a sequence, as shown in Equations 1.11 and 1.12. In addition, each sub-layer in the Transformer includes residual connections and layer normalization, which help mitigate the vanishing gradient problem in deep models and improve training stability and convergence speed.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (1.11)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (1.12)$$

Owing to its powerful feature extraction abilities and efficient parallel processing, the Transformer has demonstrated strong performance in natural language processing tasks such as machine translation [19], text summarization [20], and question answering [21]. As research has progressed, it has become clear that the Transformer's robust

global modeling and feature learning capabilities also make it highly suitable for visual tasks.

Compared to CNNs, the Transformer shows significant advantages in image processing. While CNNs are effective at extracting local features, capturing long-range dependencies requires stacking many convolutional layers to enlarge the receptive field, which greatly increases model parameters and computational cost. In contrast, the self-attention mechanism in Transformers can directly capture relationships between any positions in the image, regardless of distance, in a single step. This leads to more efficient and accurate modeling of long-range dependencies in visual data.

Vision Transformer (ViT) [22] is an innovative model that first successfully applied the Transformer architecture to image classification. Proposed by Google Research in 2020, ViT treats an image as a sequence of data. Specifically, it divides an input image into fixed-size patches (e.g., 16×16 pixels), flattens each patch into a 1D vector, and maps it to a fixed-dimensional embedding space. Positional encoding is then added to these patch embeddings to preserve spatial information. The resulting sequence is fed into a standard Transformer encoder, where multi-layer multi-head self-attention is used to model global relationships between patches, as illustrated in Figure 1.11.

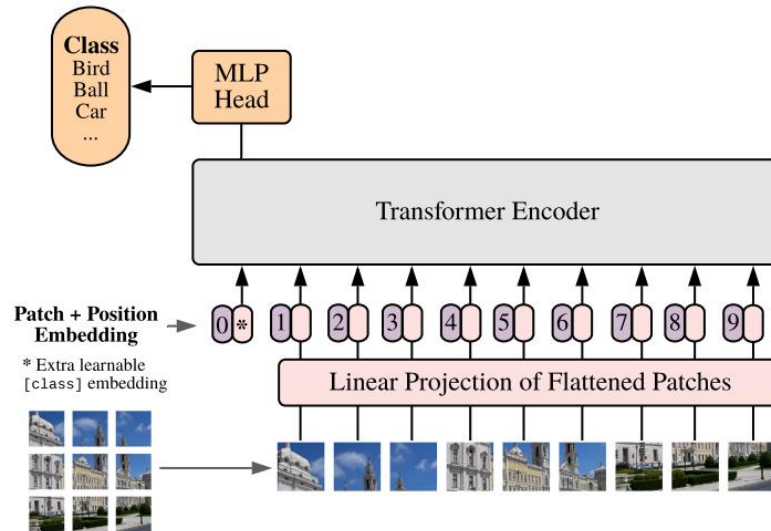


Figure 1.11 The architecture of ViT [22] .

This design enables ViT to directly model the dependencies between any two patches in an image, regardless of their distance, using a single attention step. Due to its efficient training enabled by the Transformer’s parallel architecture, ViT performs particularly well when applied to large-scale image classification tasks. Experimental results show that on large datasets like ImageNet-21k, ViT significantly outperforms traditional CNN models.

However, ViT also faces certain challenges. Due to the lack of built-in local inductive biases, such as translation invariance and locality, which are inherent in CNNs, ViT often performs worse than CNNs on small or medium-sized datasets. Moreover, the computational complexity of self-attention in ViT grows quadratically with the length of the input sequence, making it computationally demanding for high-resolution images.

Despite these limitations, the success of ViT paved the way for applying transformer architectures to vision tasks. It inspired the development of many Transformer-based vision models, such as DeiT [23] and Swin Transformer [24] , which retain ViT’s global modeling capability while improving computational efficiency and

multi-scale feature learning. These advances have expanded the range of applications for Transformers in computer vision.

Data-efficient Image Transformer (DeiT), proposed by Facebook AI Research in 2021, is an efficient vision Transformer architecture designed to reduce ViT's reliance on large datasets. One of its key innovations is the introduction of a distillation token, a special token created for knowledge distillation. This allows the model to learn image classification tasks while effectively transferring knowledge from a CNN teacher model, as shown in Figure 1.12.

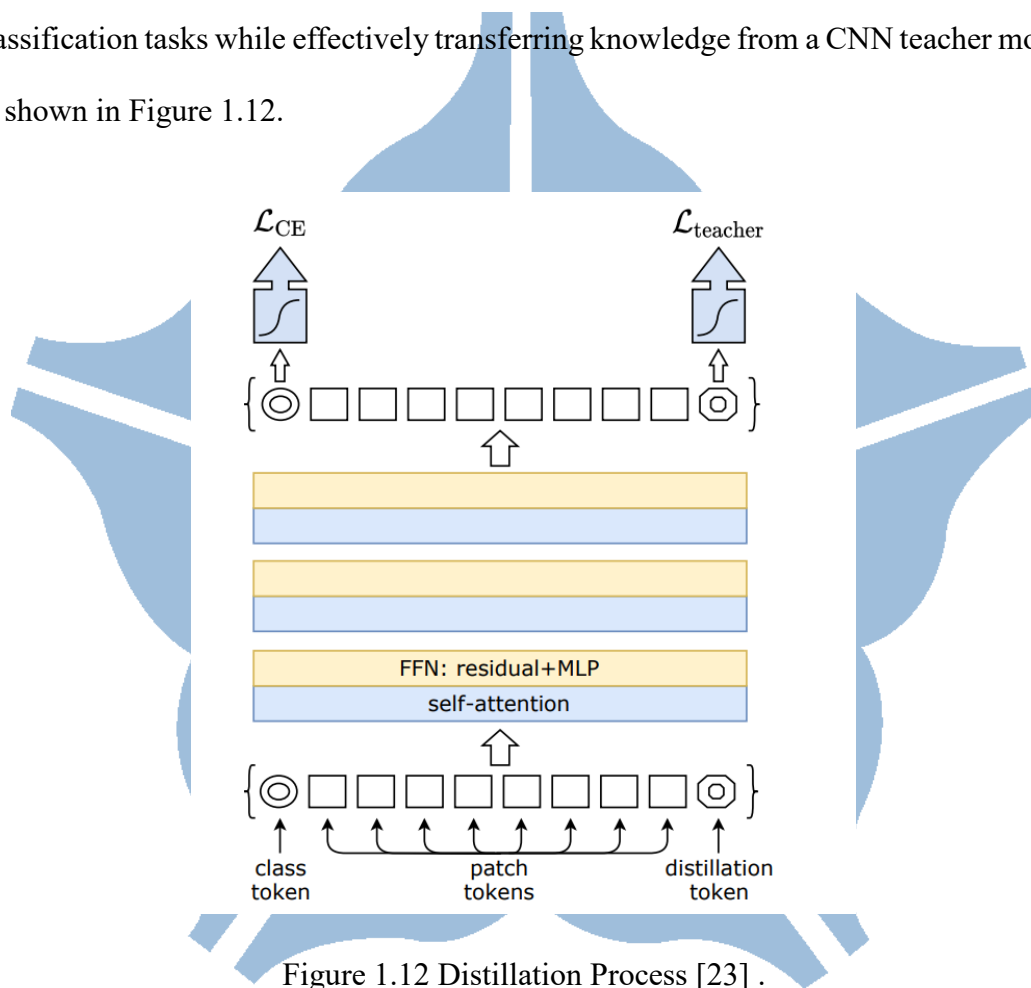


Figure 1.12 Distillation Process [23] .

Shifted Window Transformer (Swin Transformer) is a hierarchical vision Transformer architecture proposed by Microsoft Research Asia in 2021. While the original ViT has strong global modeling capabilities, it suffers from high computational cost and memory usage when handling high-resolution images. To address this issue, Swin Transformer introduces a shifted window mechanism combined with a

hierarchical design, which significantly improves model efficiency and adaptability.

Swin Transformer divides the input image into non-overlapping local windows and performs self-attention computation within each window, greatly reducing computational complexity. To overcome the limitation of isolated information caused by local windows, the model shifts the window positions between adjacent layers, enabling effective feature interaction across windows, as illustrated in Figure 1.13.

In addition, the Swin Transformer employs a hierarchical feature pyramid structure akin to that used in CNNs, enabling it to effectively capture multi-scale features, as shown in Figure 1.14. This architectural design makes it particularly suitable for dense prediction tasks such as object detection and semantic segmentation.

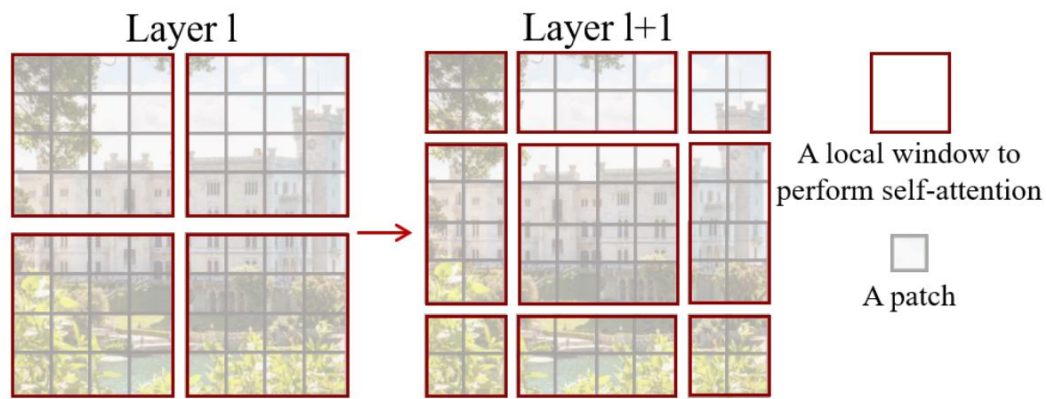


Figure 1.13 An illustration of the shifted window approach [24] .

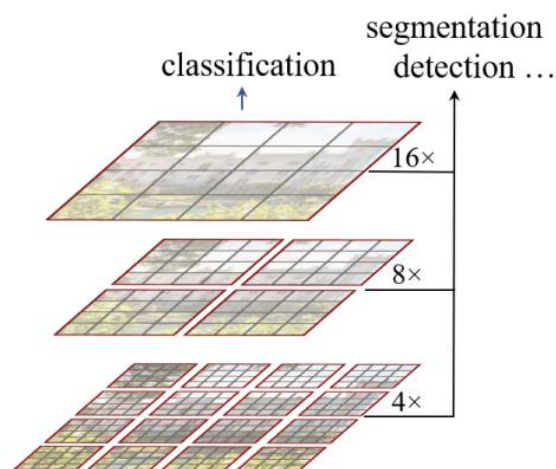


Figure 1.14 Hierarchical feature maps [24] .

1.4 Lightweight Transformers for Mobile Devices

In recent years, the Transformer architecture has achieved remarkable success in the field of computer vision, thanks to its strong global modeling capability and highly parallelizable computation. However, conventional vision Transformers typically involve large numbers of parameters and high computational demands, making them difficult to deploy on mobile and edge devices with limited resources and power constraints.

To make Transformer models more practical in resource-limited environments, researchers have proposed several lightweight architectures designed for these scenarios. This section explores these mobile-friendly Transformer models in detail.

Chen et al. [25] proposed a novel architecture called MobileFormer, which integrates MobileNet and Transformer through a parallel design. The model introduces a lightweight two-way bridge mechanism to enable efficient interaction between local features and global representations, as illustrated in Figure 1.15. The Transformer component uses only a small number of learnable tokens, significantly reducing computational cost. In addition, the architecture incorporates dynamic ReLU to enhance feature representation capability.

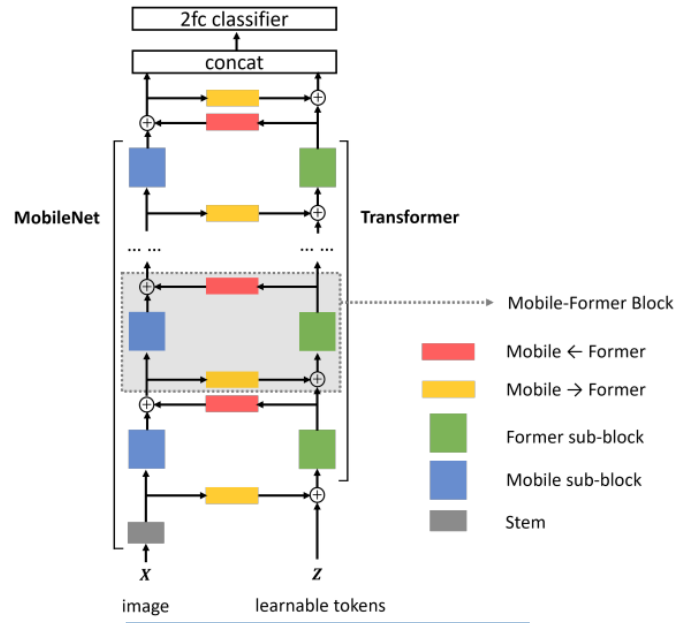


Figure 1.15 Overview of Mobile-Former [25] .

Mobile Vision Transformer (MobileViT) [26], is a lightweight vision Transformer architecture proposed by the Apple research team in 2022. It is designed to combine the global modeling capabilities of Transformers with the local feature extraction strengths of CNNs, aiming to meet the dual requirements of efficiency and accuracy for mobile and edge devices. Unlike conventional Transformers, MobileViT adopts a hybrid architecture. It first uses a lightweight convolutional network to extract local features and then applies compact Transformer modules to capture global dependencies, as illustrated in Figure 1.16.

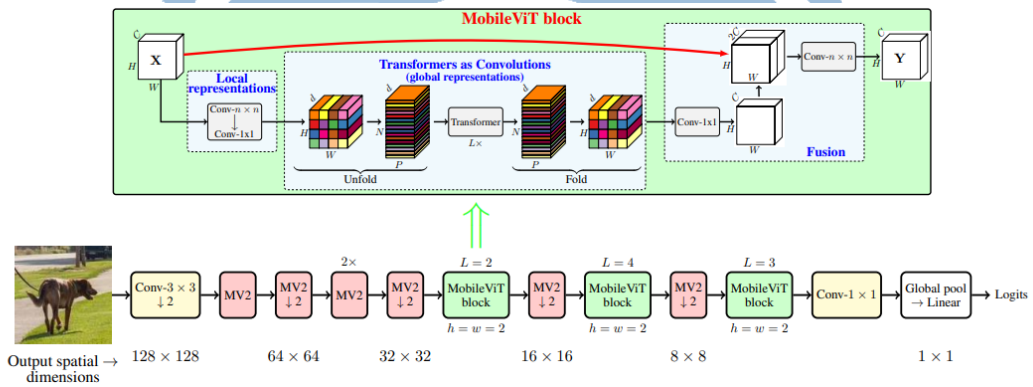


Figure 1.16 The architecture of MobileViT [26] .

In the MobileViT block, the structure is divided into three stages. First, during the Local Representations stage, the input feature map X is processed through a standard $n \times n$ convolution to extract local spatial information. This is followed by a 1×1 pointwise convolution that increases the number of channels, laying a strong foundation for the subsequent Transformer operations.

Next is the Transformers as Convolutions stage. The feature map is unfolded into N non-overlapping patch sequences. For each spatial position P , the corresponding patch sequence is passed through L Transformer layers to perform self-attention and model global dependencies across patches. Unlike traditional ViT, MobileViT preserves the spatial order of pixels within each patch, introducing spatial inductive bias into the global modeling process.

Finally, in the Fusion stage, the output sequence from the Transformer is folded back to its original spatial dimensions. A 1×1 pointwise convolution is then applied to reduce the number of channels to the original size. This result is concatenated with the original input feature map to form a 2C-dimensional fused representation, which is then passed through a final $n \times n$ convolution for integration, producing the final feature map.

The overall structure allows each pixel to retain local features while also capturing global semantic context through the Transformer, enabling effective feature modeling and fusion that enhances performance in classification and recognition tasks.

MobileViT not only offers excellent parameter efficiency, delivering competitive performance under limited computational budgets, but also provides good scalability and generality, making it suitable for various image classification and object detection tasks on mobile devices. Its architectural design has also inspired the development of subsequent lightweight Transformer models, advancing the adoption of Transformers in resource-constrained environments.

1.5 Introduction to related work

According to the findings reported in [8], the authors selected the VGG model as one of the baseline methods, which is illustrated in Figure 1.17, to assess its performance in the UAV collision avoidance task. The VGG model extracts deep features by stacking small convolutional filters and pooling layers, enabling it to effectively detect potential collisions between UAVs and obstacles in the environment.

In this study, the VGG model was initialized with pretrained weights from ImageNet and then fine-tuned on the ColANet dataset to adapt to the UAV collision prediction scenario. Experimental results demonstrated that VGG maintained good recognition performance in complex environments, highlighting its potential in collision avoidance applications.

However, the computational efficiency of the VGG model remains relatively low. Future work should consider adopting more lightweight architectures to improve deployment efficiency on resource-constrained platforms.

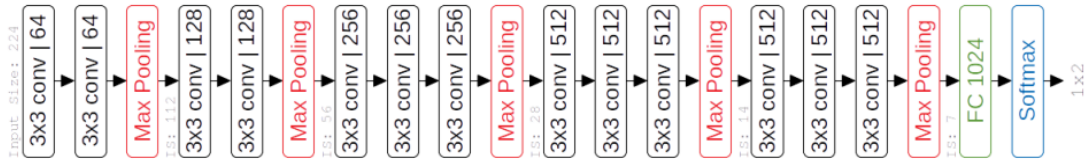


Figure 1.17 The architecture of VGG16 [8] .

As described in [27], the authors proposed a Dynamic Collision Avoidance (DCA) architecture that integrates CNNs and Recurrent Neural Networks (RNNs) to address the challenge of dynamic obstacle avoidance. The framework initially employs the lightweight MobileNetV2 model to extract feature vectors from a sequence of input images. These features are subsequently processed by a stack of three Long Short-Term Memory (LSTM) layers to capture temporal dependencies in the image sequence, enabling the system to track the motion patterns of dynamic objects. A feedforward

neural network (FNN) then performs the classification task, determining whether the current scene poses a potential collision risk, as illustrated in Figure 1.18.

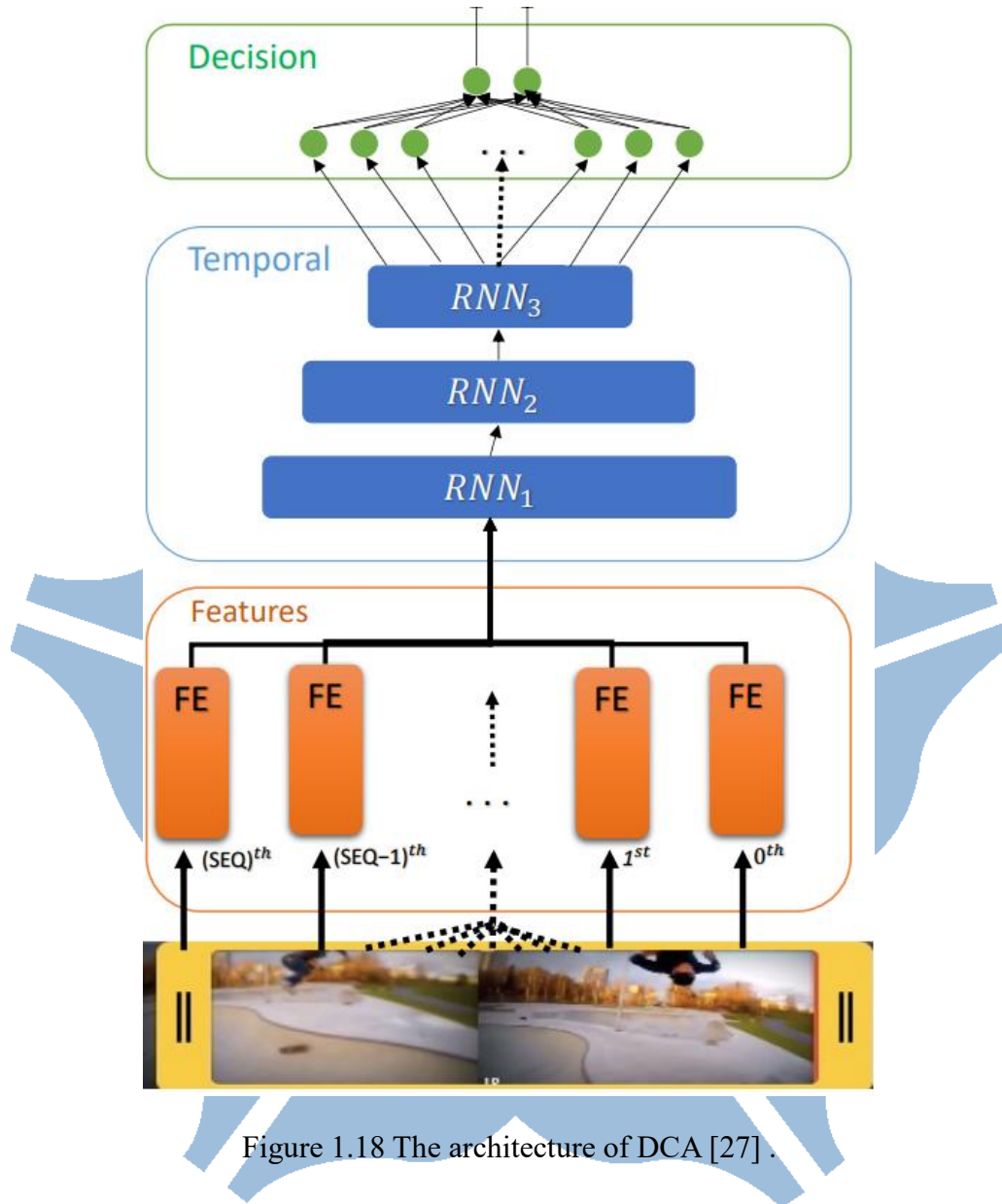


Figure 1.18 The architecture of DCA [27].

By combining real-time visual features with temporal context, the DCA approach enhances responsiveness to sudden dynamic obstacles. However, RNNs, including LSTM, are inherently sequential, with each time step depending on the hidden state of the previous one. This sequential nature prevents parallelization and causes the inference time to increase linearly with the length of the input sequence.

When deployed on UAVs or other edge devices, such a processing constraint

results in higher latency as the input sequence grows. In real-time dynamic obstacle avoidance scenarios, where rapid response is essential, the increased delay poses a significant limitation. The inability to execute timely evasive maneuvers may undermine both flight safety and the practical viability of the system.

In reference [28] , the authors developed a lightweight deep learning model for UAV collision prediction, specifically targeting resource-constrained edge devices. The research team first conducted experiments in a desktop environment using the ColANet dataset, comparing the performance of various lightweight pretrained CNN models. Among them, MobileNetV3Small [29] achieved the best overall efficiency, striking a strong balance between model size, inference speed, and power consumption. Although its accuracy was slightly lower than that of the EfficientNet series, it offered the most favorable trade-off for edge deployment.

As shown in Figure 1.19, the MobileNetV3 block adopts a lightweight convolutional design. It begins with a 1×1 pointwise convolution followed by a non-linear activation (NL) to reduce computation cost. This is followed by a 3×3 depthwise convolution to extract spatial features, and then another 1×1 convolution to integrate channel information.

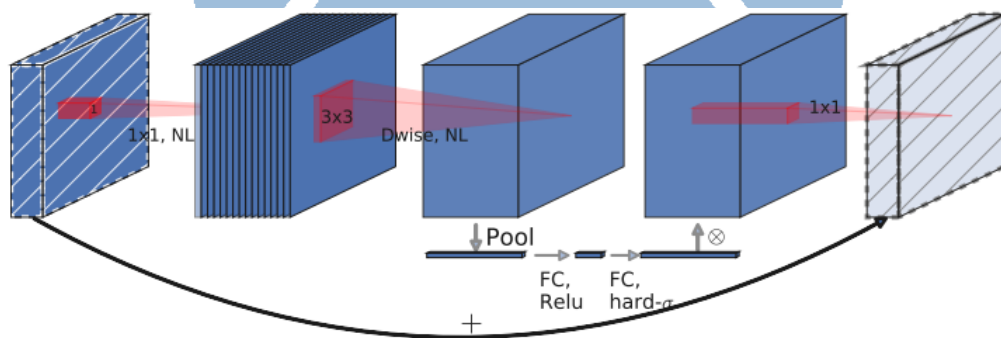


Figure 1.19 The MobilenetV3 block [29] .

In addition, a lightweight attention mechanism is introduced. After global average pooling, the output is passed through two fully connected (FC) layers (with ReLU and

hard-swish activations) to reweight the channel responses. Finally, a residual connection is added by combining the output features with the original input, which enhances feature representation while maintaining computational efficiency.

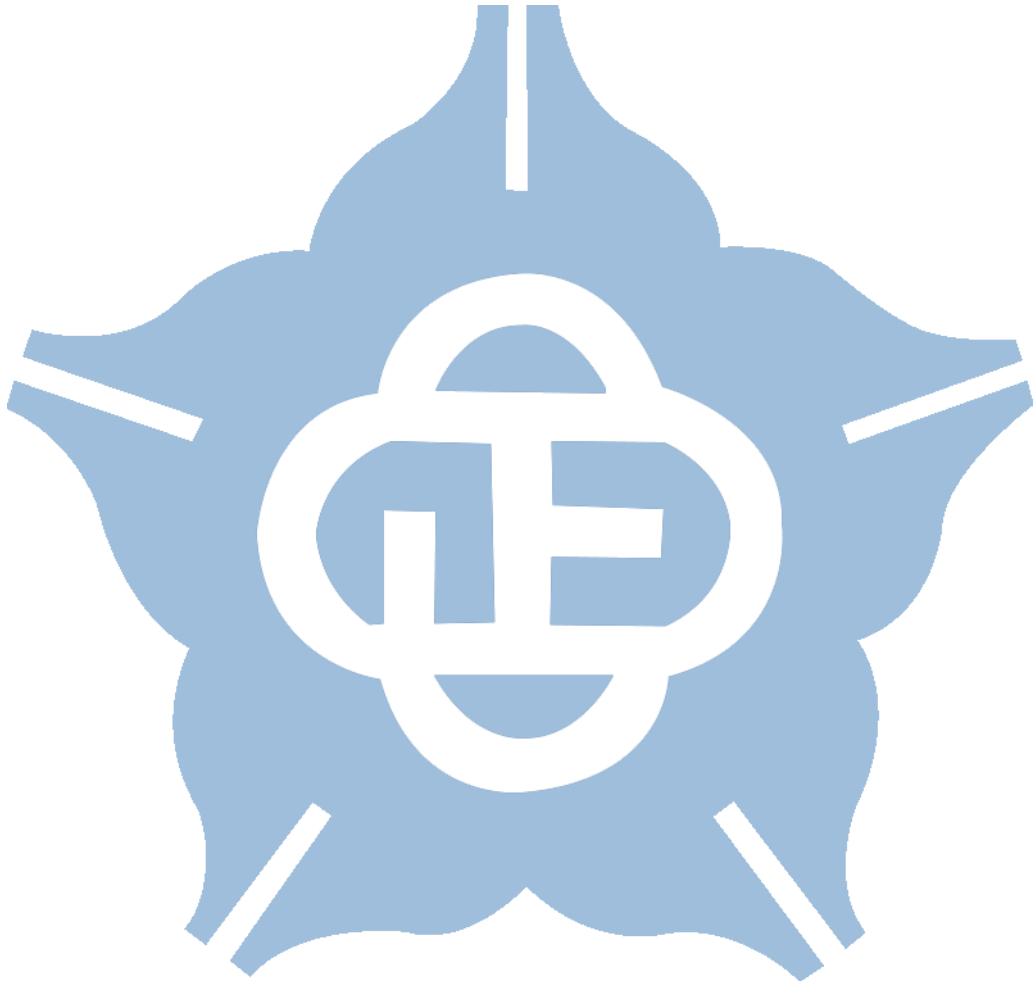
To further optimize the model, the authors utilized TensorFlow Lite (TFLite) for model conversion and applied two post-training quantization techniques: Float-16 and dynamic range quantization. Results showed that the dynamic range quantized model reduced the model size by approximately 73%, achieved the fastest inference speed, and required the lowest power consumption. Ultimately, the TFLite dynamic range quantized version of MobileNetV3Small was deployed on a Raspberry Pi 3B+, paired with real-time video input from a DJI Tello UAV. The deployment maintained the original model's accuracy while successfully achieving low power consumption and low latency, demonstrating the practical advantages and potential of lightweight models in real-world UAV collision prediction applications.

Despite the deployment efficiency of MobileNetV3Small—including low parameter count, fast inference, and minimal power usage—its test accuracy remained at only 77%–78%. This limitation is primarily due to the model's highly compressed and simplified architecture, which trades off model capacity and expressive power to reduce computational cost. As a result, it may struggle to accurately classify complex or borderline samples.

In summary, existing lightweight CNN models offer excellent spatial inductive bias and efficient inference, but remain limited in global feature modeling. In contrast, Vision Transformers (ViT) and their variants are capable of capturing long-range dependencies, yet their large parameter sizes and strong reliance on data augmentation make them less suitable for deployment on resource-limited mobile devices. Although several recent hybrid architectures have attempted to combine the strengths of CNNs and Transformers, many still face challenges such as excessive architectural complexity

or limited training stability.

Therefore, the present study aims to develop a model that balances energy efficiency and lightweight design, integrating the local perception capabilities of CNNs with the global modeling strengths of Transformers. The proposed solution is tailored for UAV collision avoidance tasks, offering both practical applicability and competitive performance.



1.6 Hardware-Friendly Softmax

Approximations in Transformer

As previously discussed, one of the core components of the Transformer architecture is the multi-head self-attention mechanism, which relies heavily on the Softmax function. Although Softmax is theoretically simple, it presents considerable challenges in hardware implementation. As shown in Equation 1.13, it involves exponential and division operations that are computationally expensive and particularly difficult to implement efficiently on resource-constrained platforms such as FPGAs or ASICs.

$$\text{Softmax}(x) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad (1.13)$$

Before exploring hardware-level optimizations, it is essential to gain a deeper understanding of the Softmax computation process and examine the associated design considerations when implementing it in hardware accelerators. This analysis is critical not only for identifying performance bottlenecks in Transformer architectures, but also for laying the foundation for targeted optimization strategies.

Common approximation methods used to simplify Softmax computation include lookup tables [30], the Coordinate Rotation Digital Computer (CORDIC) algorithm [31], base substitution techniques [32], and Taylor series expansion [33].

In reference [30], Wang et al. proposed LUTNet, a deep neural network acceleration framework specifically designed for FPGA deployment. This framework innovatively replaces the traditional XNOR units in Binarized Neural Networks (BNNs) with higher-density K-LUTs, leveraging the native lookup table (LUT) architecture of FPGAs to implement complex Boolean operations during inference. These operations approximate the multiplication-accumulation and Softmax functions found in

conventional neural networks. By mapping neural network computations to arbitrary Boolean functions within LUTs, LUTNet preserves the non-linear characteristics of Softmax while significantly improving hardware logic density and energy efficiency.

Figure 1.20 illustrates the core transformation in the LUTNet architecture. In a typical BNN, inputs and weights are processed through XNOR operations followed by accumulation. In LUTNet, this is replaced by multiple 2-LUT units, which flexibly combine input features to perform more expressive logic mappings. This transformation allows effective approximation of non-linear functions such as Softmax, and when combined with pruning techniques, redundant units can be removed to further reduce logic resource usage. Finally, a summation unit integrates the multi-path outputs to produce the final inference result.

The proposed architecture fully exploits the inherent advantages of FPGAs, achieving higher area efficiency and energy efficiency compared to conventional BNN implementations.

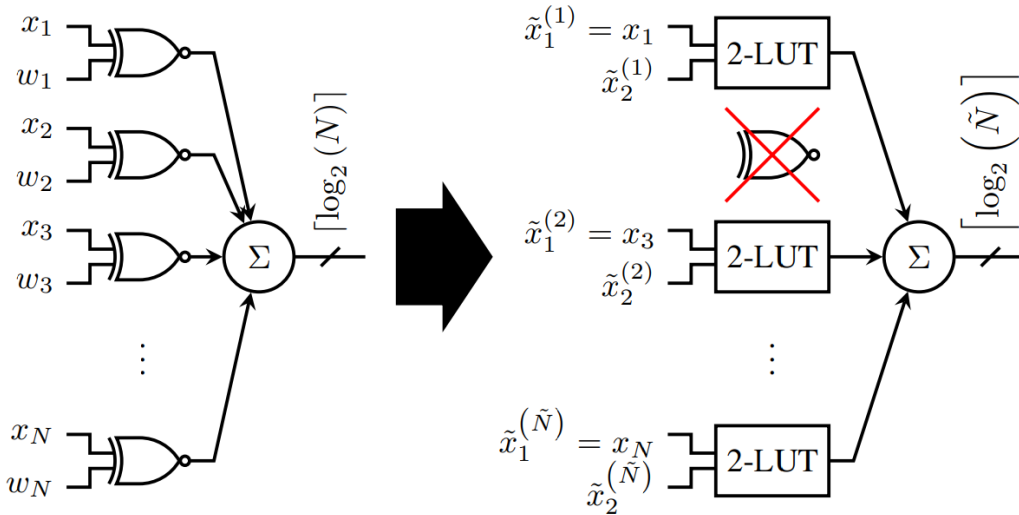


Figure 1.20 BNN to LUTNet architectural transformation for a single channel [30] .

In reference [31] , the authors proposed a hardware design framework for the Softmax function based on the CORDIC algorithm, as illustrated in Figure 1.21. The

overall design is divided into two main components. First, a CORDIC EXP unit is used to compute the exponential operations required by Softmax. The CORDIC algorithm relies solely on comparisons (CMP), shifts (SHIFT), and additions/subtractions (ADD/SUB), thereby avoiding the need for hardware multipliers and dividers and enabling a more lightweight implementation.

Second, for the division operation involving the accumulation in the Softmax denominator, the design offers two options: one uses a CORDIC DIV unit, which continues to follow the CORDIC-based computation logic; the other uses a binary division algorithm, which performs division through iterative subtraction.

By integrating these two computational stages, the framework completes the full CORDIC-based Softmax computation flow. This design significantly improves hardware computational efficiency and reduces resource consumption, making it well-suited for edge computing platforms and neuromorphic chips, where resource constraints are critical.

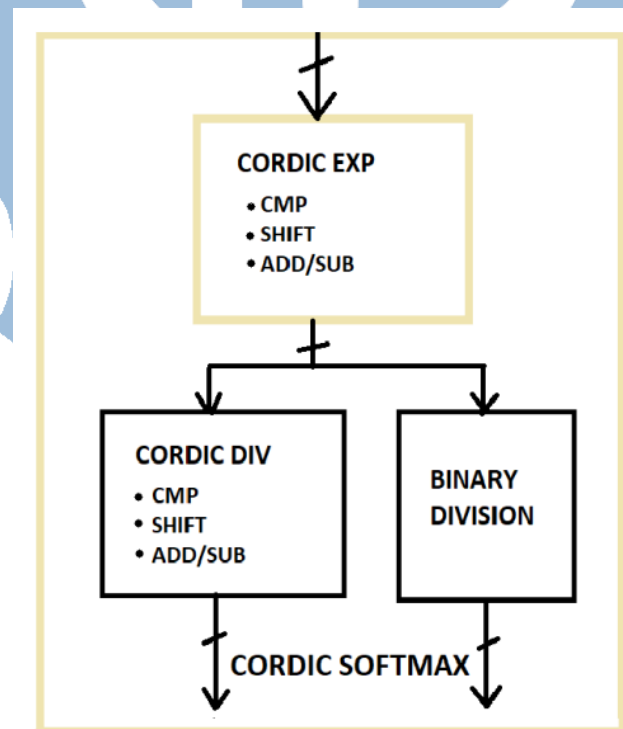


Figure 1.21 CORDIC Softmax [31] .

In reference [32] , the authors proposed a Base-2 Softmax function as an alternative to the traditional natural exponential (base-e) Softmax, aiming to address the high resource consumption and low efficiency typically encountered in hardware implementations. The formulation is shown in Equation 1.14. Through mathematical derivation and software simulations, it was demonstrated that the Base-2 Softmax retains essential mathematical properties such as normalization and the simplification of cross-entropy derivatives, making it suitable for neural network training.

$$\text{Softmax}(x) \approx \frac{2^{x_i}}{\sum_{j=1}^n 2^{x_j}} \quad (1.14)$$

To further enhance hardware efficiency, the study introduces a Symmetric-Mapped Lookup Table (SM-LUT) architecture, designed to be both low in complexity and high in accuracy. As shown in Figure 1.22, the SM-LUT exploits the symmetry between exponential and logarithmic operations, using two lookup tables, LUT1 and LUT2, in conjunction with a mapping mechanism that relates input values to result addresses. When the control signal $c=0$, SM-LUT is used for exponential calculation; when $c=1$, it is used for logarithmic calculation.

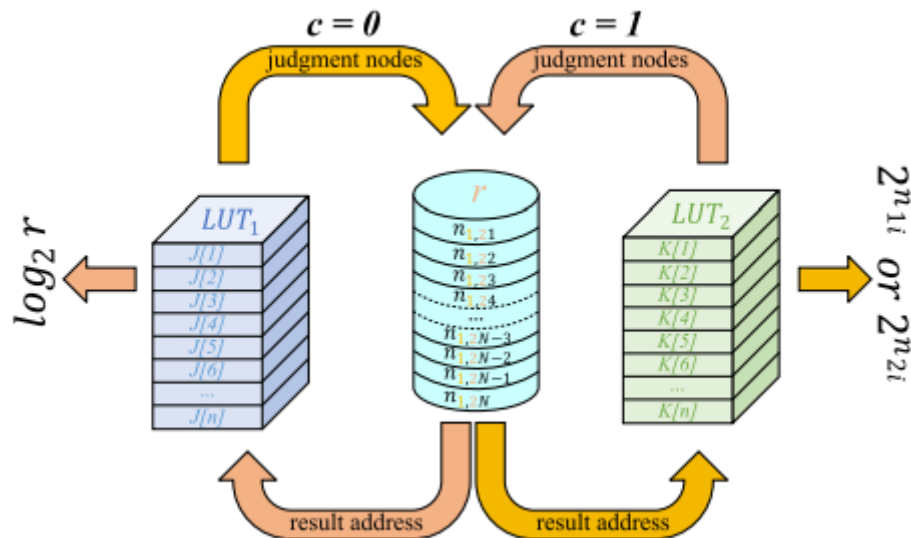


Figure 1.22 Hardware architecture of SM-LUT module [32] .

The proposed design significantly reduces hardware resource usage while accelerating computation. Compared to existing methods, the proposed architecture achieves optimized accuracy and hardware performance, offering a more efficient Softmax hardware solution for deep learning accelerators.

In reference [33], to reduce the hardware complexity of exponential computations in the Softmax function, the authors used a Taylor series expansion to approximate the exponential function. As shown in Equation 1.15, this simplifies complex exponential operations into polynomial form, facilitating more efficient computation. For low-bit input data, a first-order Taylor approximation (Equation 1.16) allows the exponential to be computed using only addition and basic arithmetic operations, eliminating the need for high-order multiplications and divisions. The method significantly reduces hardware resource consumption and improves computational efficiency.

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \quad (1.15)$$

$$e^x \approx 1 + x \quad (1.16)$$

The proposed method also incorporates a piecewise lookup table design, balancing both accuracy and efficiency. As a result, it stands out as one of the key techniques in this study for achieving an efficient hardware implementation of the Softmax function.

1.7 Quantization Methods in Neural Network

In traditional designs, deep neural networks (DNNs) are typically trained and inferred using 32-bit floating-point precision. While this ensures strong learning capability and high prediction accuracy, it also results in heavy computational workloads and longer execution times. For resource-constrained edge devices, such designs often impose significant burdens on hardware.

To address this issue, quantization techniques have been introduced. Quantization reduces the numerical precision of model weights and activations, thereby significantly lowering both computational complexity and memory requirements, while striving to maintain acceptable prediction accuracy. This leads to improved overall efficiency in edge deployments.

A prominent example is the Binarized Neural Network (BNN), which restricts weights to only two discrete values, typically +1 or -1 (as shown in Equation 1.17). Such a design dramatically reduces storage demands and accelerates computation. However, such simplification comes at the cost of reduced model expressiveness, and often results in lower accuracy compared to models using higher-precision weights.

$$x_b = \text{sign}(x) = \begin{cases} 1, & \text{if } x > 0 \\ -1, & \text{otherwise} \end{cases} \quad (1.17)$$

In contrast, the Ternary Weight Network (TWN) offers a compromise between accuracy and efficiency. In TWNs, weights are constrained to three possible values: +1, 0, or -1 (as shown in Equation 1.18). This ternarization approach reduces computational complexity and model size while retaining higher accuracy compared to fully binarized models.

$$W_i^t = \begin{cases} +1, & \text{if } x > \Delta \\ 0, & \text{if } x \leq \Delta \\ -1, & \text{if } x < 0 \end{cases} \quad (1.18)$$

Although BNNs and TWNs effectively reduce computational and storage requirements by compressing numerical precision to 1-bit and 2-bit, respectively, there remains a noticeable accuracy gap compared to traditional high-precision models. To address the limitations in quantization flexibility, DoReFa-Net [34] proposed a method that allows both weights and activations to be quantized to arbitrary bit-widths. This approach overcomes the rigid constraints of previous BNN and TWN designs. Equation 1.19 describes how an input value x is transformed into a k -bit representation.

For low-bit weight quantization, Equation 1.20 defines the process by first normalizing weights using the maximum absolute value from the current layer. To ensure the quantized weights remain within the $[-1,1]$ range, a $\tanh$ function is applied, constraining values to be compatible with k -bit fixed-point formats.

$$quantized_k(x) = \frac{1}{2^{k-1}} \text{round}((2^k - 1)x) \quad (1.19)$$

$$f_w^k(r) = 2quantized_k\left(\frac{\tanh(r)}{2\max(|\tanh(r)|)} + \frac{1}{2}\right) - 1 \quad (1.20)$$

As for activations, the values must first be mapped to the range $[0,1]$ using a bounded function, as shown in Equation 1.21. These normalized values are then quantized into k -bit representations according to Equation 1.22.

$$h(x) = clip(x, 0, 1) \quad (1.21)$$

$$f_a^k(r) = quantized_k(r) \quad (1.22)$$

In reference [35], the authors proposed an efficient post-training quantization (PTQ) method for the Longformer model, aiming to reduce computational complexity and model storage requirements during inference. Longformer utilizes a sliding window attention mechanism, which enables the efficient handling of long text inputs compared to standard Transformer architectures. However, despite its improvements, large-scale

models like Longformer still face challenges related to high computational costs and memory usage when deployed, making quantization a critical technique.

In the proposed quantization framework, the authors adopted a symmetric quantization scheme, where both the input activations and weights of model matrices are scaled and discretized. Equations 1.23 and 1.24 describe how the input is quantized using a scaling factors.

$$Q_1 = \left\| \frac{x_1 * 2^{b-1} - 1}{\text{clipping_range}(x_1)} \right\| = \|x_1 * s_1\| \quad (1.23)$$

$$Q_2 = \left\| \frac{x_2 * 2^{b-1} - 1}{\text{clipping_range}(x_2)} \right\| = \|x_2 * s_2\| \quad (1.24)$$

To further reduce quantization error caused by outliers, the authors proposed an optimized percentile method based on Mean Squared Error (MSE) minimization. Specifically, Equation 1.25 defines how to select a percentile value as the clipping upper bound r . Then, Equation 1.26 formalizes the clipping operation, followed by Equation 1.27, which calculates the MSE between the original matrix and the quantized matrix.

$$r = \text{percentile}(X, p) \quad (1.25)$$

$$\text{clip}(x) = \begin{cases} -r, & \text{if } x < -r \\ x, & \text{if } -r \leq x \leq r \\ r, & \text{if } x > r \end{cases} \quad (1.26)$$

$$\text{mse}(X) = \sum (X - \frac{Q}{s})^2 \quad (1.27)$$

By searching for the optimal percentile p that minimizes the MSE, this method effectively addresses the limitations of traditional min-max clipping, which is often sensitive to outliers and results in reduced quantization resolution. The proposed approach improves quantization precision by discarding extreme values that have

negligible impact on overall model accuracy but significantly affect scaling.

Moreover, considering that Longformer's sliding window attention mechanism results in significant variation in value ranges across different windows, applying layerwise quantization can lead to suboptimal results. Specifically, extreme values in a few windows may dominate the scaling factor, thereby degrading the quantization precision of the majority. To address this issue, the authors proposed a Chunkwise Quantization strategy, which performs independent quantization on each attention window (or chunk). This prevents the overall quantization performance from being adversely affected by outliers in individual chunks.

While quantized neural networks can significantly reduce computational resource usage and improve efficiency, it is important to note that the quantization process can also introduce additional error, potentially impacting model accuracy. Therefore, selecting an appropriate quantization strategy requires careful trade-offs, taking into account factors such as task-specific requirements, architectural characteristics, and the balance between model complexity and accuracy.

1.8 Motivation

Chapter 1 presented an analysis and discussion of various UAV collision prediction methods, with particular emphasis on the integration of deep learning techniques. The application of deep neural networks significantly enhances the capability of UAVs to autonomously avoid obstacles and navigate safely in complex environments. However, the substantial computational demands of deep models continue to pose a significant challenge, given the strict constraints on power and hardware resources of UAV platforms. Thus, identifying solutions that achieve a balance between model complexity and prediction accuracy is essential.

In response to these challenges, the present study proposes an efficient and lightweight collision prediction model tailored for UAVs operating under resource constraints. Leveraging deep learning techniques, the proposed architecture achieves a balanced compromise between predictive accuracy and computational efficiency. By combining the strengths of CNNs and Transformers, the model effectively addresses both local feature extraction and long-range dependency modeling. The hybrid design minimizes computational overhead and enhances inference efficiency.

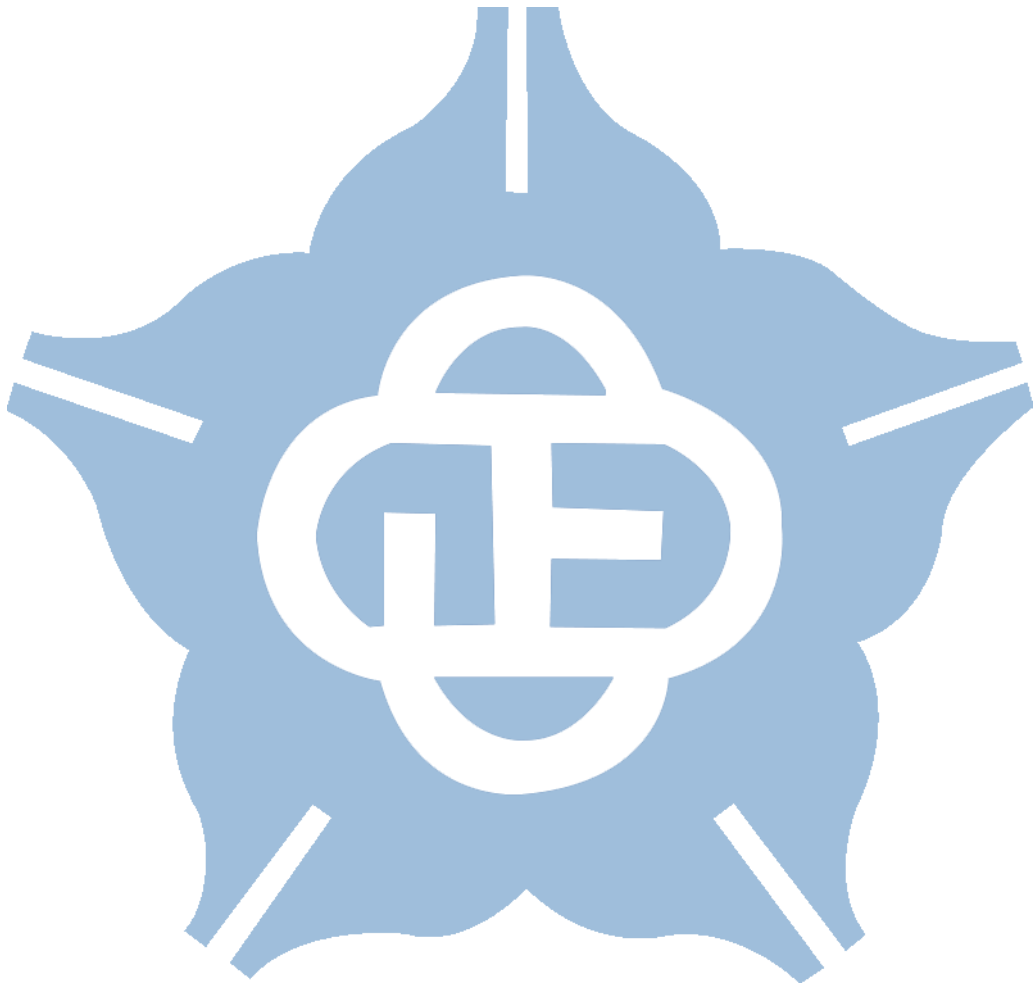
To further improve hardware feasibility, quantization techniques were applied, enabling the model to operate at reduced bit precision and thereby significantly decreasing memory usage and power consumption. This enhancement facilitates more practical and efficient deployment on UAV platforms in real-world scenarios.

The main contributions of study are as follows:

1. A hybrid architecture that fuses CNN-based local feature extraction with Transformer-based global dependency modeling, leading to improved collision prediction accuracy and overall performance.
2. A hardware-friendly Transformer design through streamlined multi-head self-

attention, ensuring suitability for resource-constrained edge devices.

3. Real-time inference capability enabled by model simplification and computational pipeline optimization, significantly reducing latency for practical UAV applications.



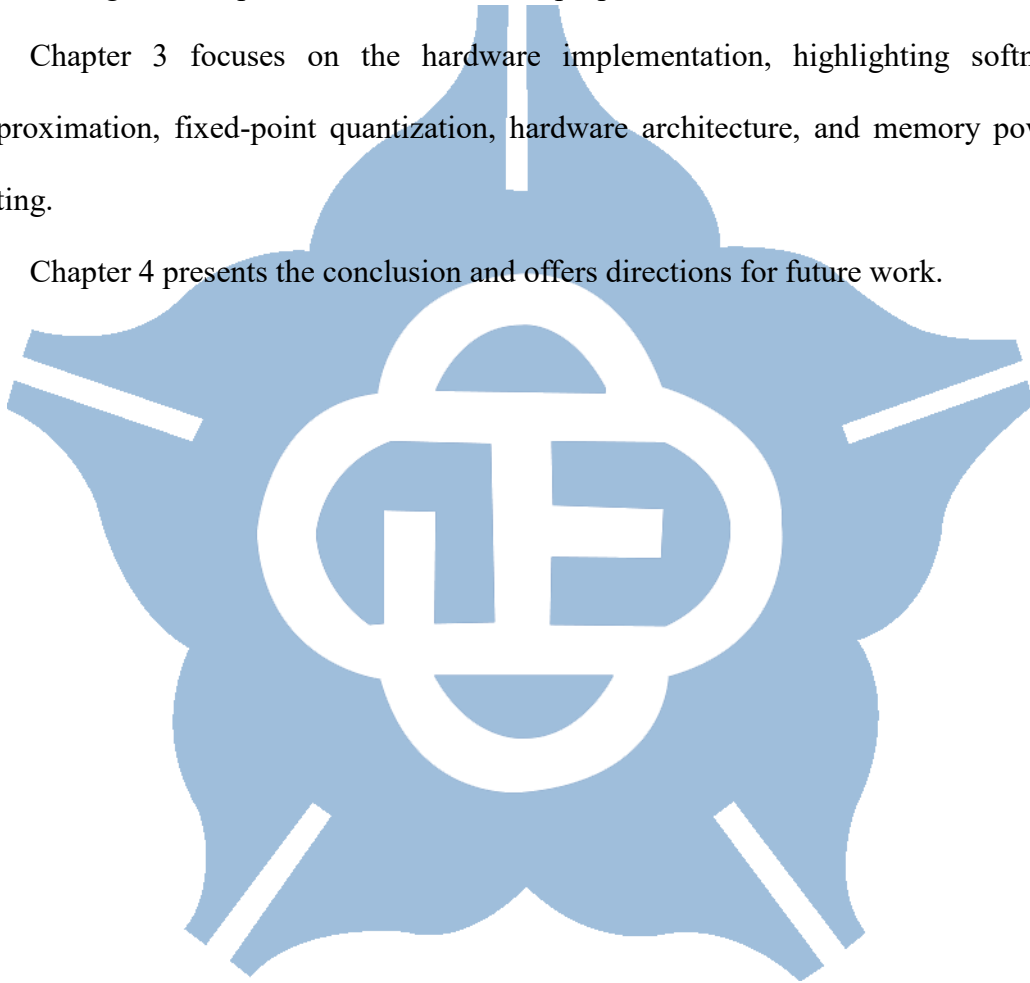
1.9 Thesis Organization

Chapter 1 provides an overview of UAV collision avoidance, CNN and Transformer architectures, lightweight Transformer designs, related works, quantization techniques, and the motivation for this study.

Chapter 2 details the software implementation, including dataset preprocessing, model design, and experimental results of the proposed CFVT model.

Chapter 3 focuses on the hardware implementation, highlighting softmax approximation, fixed-point quantization, hardware architecture, and memory power gating.

Chapter 4 presents the conclusion and offers directions for future work.



Chapter 2 Software Implementation

2.1 Architecture Overview

In this study, a UAV collision prediction method is proposed. As described in [8], the dataset used consists of two categories: collision and normal flight, as illustrated in Figure 1.1. A total of 3,120 images depicting collision scenarios and 15,752 images of normal flight were collected. After random shuffling, 80% of the dataset was allocated for model training and the remaining 20% for testing. The specific number of samples for each category in the training and testing sets is detailed in Table 2.1.

Table 2.1 The amount of training set and test set in each class

Class	Amount of train data	Amount of test data
Collision	2494	626
Normal flight	12,603	3149
Total	15,097	3775

As illustrated in Figure 2.1, the model development process begins with selecting an appropriate image preprocessing method. This is followed by designing a lightweight model with the relatively low parameters, which is then trained using the PyTorch. Given that the current state-of-the-art accuracy on the dataset is approximately 95%, and considering the potential drop in accuracy due to subsequent model quantization, the model must first achieve at least 98% accuracy at this stage to ensure the feasibility of the following steps.

Next, the computational and memory constraints of edge devices must be taken into account in selecting an appropriate quantization strategy. The goal is to strike an optimal balance between model accuracy and model size. Only when the accuracy drop

after quantization is confirmed to be less than 1% can the process proceed to the next step.

Finally, the entire model pipeline must be implemented in Python to ensure feasibility for deployment on the target hardware platform. Once all steps have been completed and verified, the proposed model can then be deployed and validated on hardware.

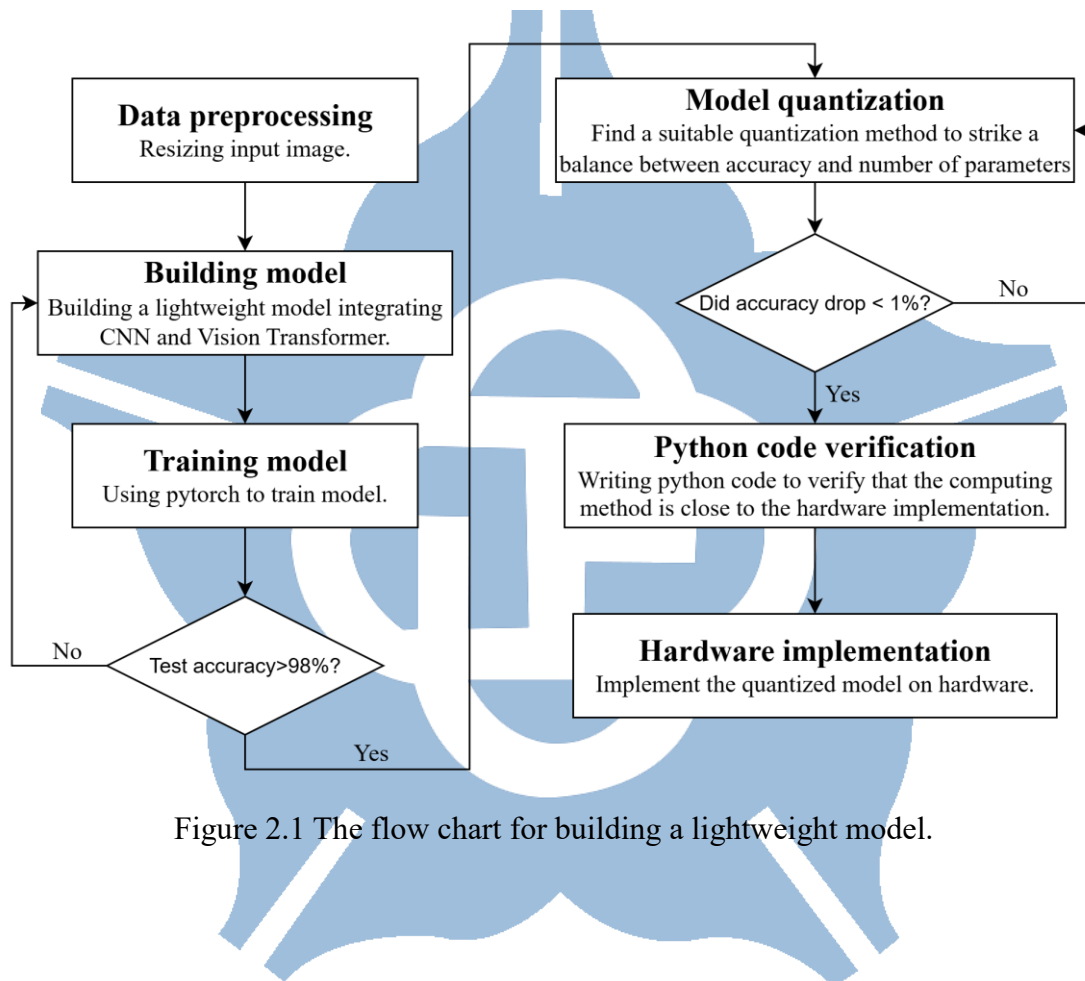


Figure 2.1 The flow chart for building a lightweight model.

2.2 Dataset Preprocessing

When selecting an image preprocessing method, it is essential to consider both its impact on model accuracy and its computational resource requirements. In this study, all input images have a resolution of 300×300 pixels. To improve processing efficiency and meet the input size requirements of the model, the images are progressively downscaled by half in each iteration and used for training to evaluate the impact on accuracy.

Several commonly used interpolation methods, including nearest neighbor interpolation, bilinear interpolation, and bicubic interpolation, were employed to resize the images. Table 2.2 summarizes the impact of different image sizes and interpolation methods on classification accuracy.

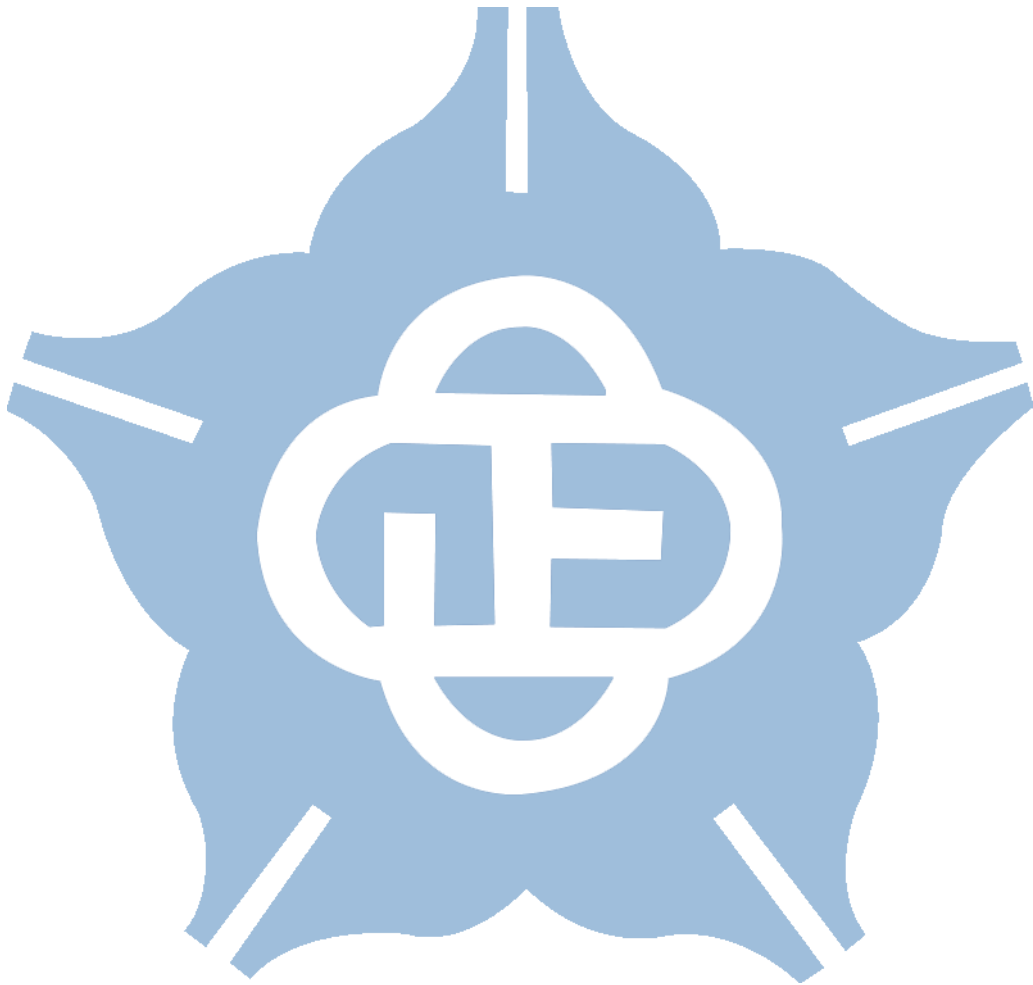
Table 2.2 The accuracy of different compression methods across various image sizes.

Image size	Nearest neighbor	Bilinear interpolation	Bicubic interpolation
300×300	98.1%	98.45%	98.55%
150×150	97.9%	98.42%	98.56%
75×75	97.81%	98.46%	98.52%
38×38	95.24%	96.41%	97.02%

The results show that when images are downscaled to 38×38 pixels, all three interpolation methods result in a significant drop in accuracy. However, at a resolution of 75×75 pixels, both bilinear and bicubic interpolation maintain relatively high accuracy, with bicubic performing slightly better. Nonetheless, bicubic interpolation incurs noticeably higher computational costs.

Considering both classification accuracy and computational efficiency, this study ultimately adopts bilinear interpolation and standardizes all input images to a resolution

of 75×75 pixels for subsequent model training and evaluation.



2.3 Evaluation and Selection of Feature Extraction Modules

In the model architecture proposed in this thesis, the front-end module is designed for local feature extraction and downsampling, thereby reducing the spatial dimensions of the input feature maps before they are passed to the Transformer module. This design contributes to lowering the computational cost and memory usage during the subsequent global modeling stage. To evaluate the compatibility between different CNN architectures and the MobileViT module, a fixed architecture was established in which only the front-end CNN module varied, while the back-end remained a single MobileViT module. The complete architecture is illustrated in Figure 2.2, and detailed information regarding the MobileViT architecture is provided in Section 1.4. This configuration enables the isolation of the impact of CNN-based feature extractors on overall model performance.

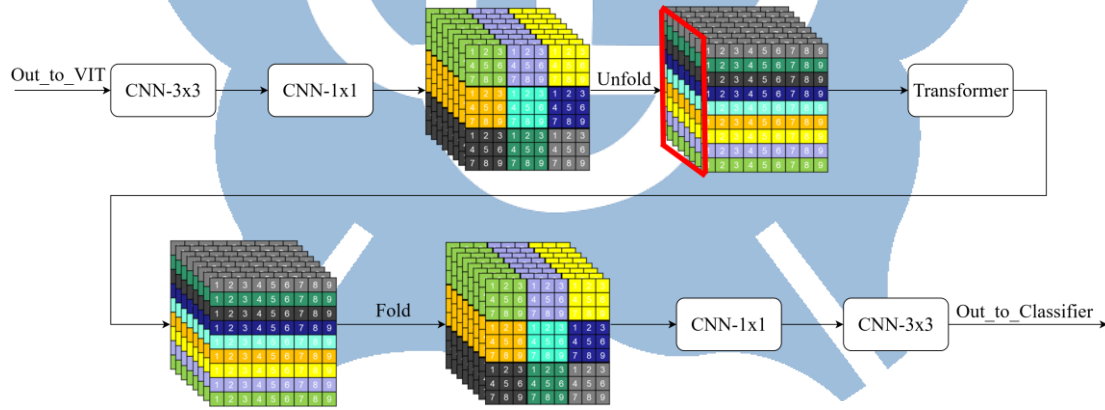


Figure 2.2 The structure of ViT block.

The candidate CNNs include custom-designed standard CNN, MobileNetV1, and MobileNetV2. When integrated with the MobileViT block, the resulting models are named CNNFusionViT(CFVT), MNV1FusionViT(MN1VT), and MNV2FusionViT (MN2VT), respectively. Their complete structures are shown in Figure 2.3, Figure 2.4

and Figure 2.5, respectively. Details of the MNV1 block and MNV2 block can be found in Section 1.2.

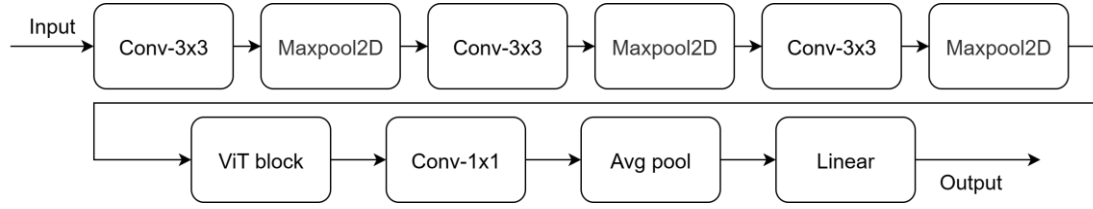


Figure 2.3 The structure of CFVT.

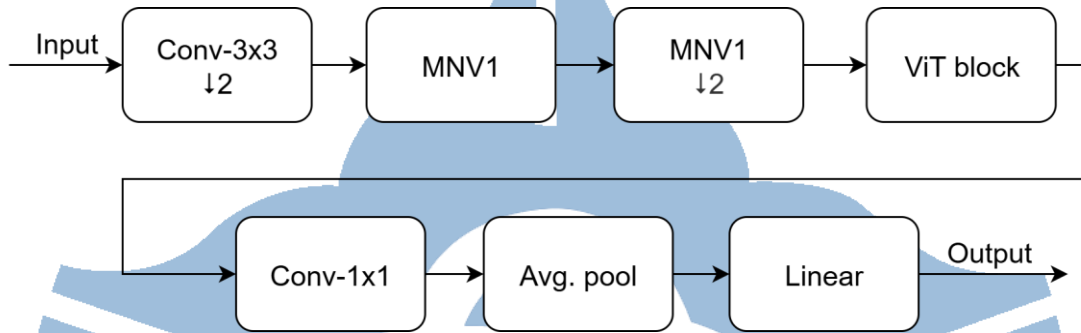


Figure 2.4 The structure of MN1VT. '↓2' indicates stride = 2; the stride is 1 by default.

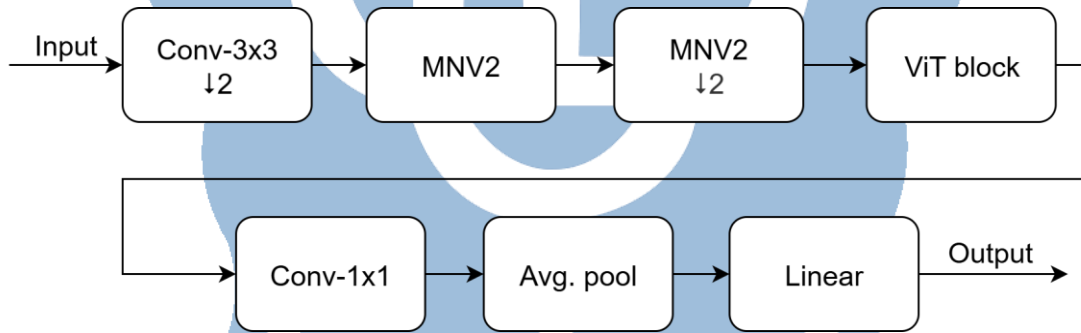


Figure 2.5 The structure of MN2VT. '↓2' indicates stride = 2; the stride is 1 by default.

As shown in Table 2.3, CFVT achieved the highest classification accuracy of 98.46%, demonstrating a significant performance improvement over MN1VT and MN2VT, although all three models exhibit comparable computational complexity.

These findings suggest that although MobileNet architectures are efficient due to their use of depthwise separable convolutions, their local feature extraction capability

is less effective in this dataset. The standard CNN proved to be the most compatible with the ViT block, offering a better synergy between local and global feature representation. As a result, the CNN was selected as the front-end module in the final architecture for subsequent training and evaluation.

Table 2.3 Performance Comparison of Different Feature Extraction Modules

Combined with ViT Block.

Method	Parameter(k)	FLOPs(M)	Accuracy(%)
MNV1FusionViT	21.63	12.92	95.68
MNV2FusionViT	25.98	16.6	93.96
CNNFusionViT	31.49	13.9	98.46

2.4 Implementation of CNNFusionViT(CFVT) and Software Analysis

Table 2.4 presents a comparison of test accuracy across different combinations of convolutional layer depths and channel sizes. To determine the most suitable CNN architecture for this study, a stepwise experimental approach was adopted. The process began with a basic three-layer convolutional network (Method 1), serving as a minimal configuration for initial evaluation. Subsequently, the number of channels was increased while keeping the number of layers unchanged (Method 2), resulting in a noticeable improvement in classification accuracy. This confirmed that increasing channel width enhances feature representation capability.

Table 2.4 The test accuracy of the CFVT model with various layers and channels.

Method	Conv.Layer(in_channel/out_channel)				Parameter(k)	FLOPs(M)	Accuracy(%)
	Layer1	Layer2	Layer3	Layer4			
1	3/8	8/12	12/24		30.05	11.44	98.12
2	3/8	8/16	16/24		31.49	13.9	98.46
3	3/8	8/24	24/32		48.12	18.88	98.07
4	3/8	8/16	16/24	24/32	51.03	22.48	88.05

To further examine the relationship between channel size and performance, Method 3 was designed by significantly increasing the number of channels. However, this led to a decline in accuracy, indicating that overly large channel configurations may introduce redundancy or overfitting, ultimately harming performance. Following this, Method 4 extended the architecture of Method 2 by adding one more convolutional layer to test whether greater depth could yield additional improvements. The results showed no meaningful gain in accuracy.

Considering the balance between accuracy, model size, and computational cost, Method 2 was selected as the backbone CNN architecture for subsequent integration with other components of the proposed model.

To assess the impact of the number of stacked ViT blocks on overall model performance, three configurations with $N = 1, 2$, and 3 blocks were evaluated. Table 2.5 presents the number of parameters, FLOPs, and test accuracy corresponding to each configuration.

Results indicate that using a single ViT block already yields the highest test accuracy of 98.46%, while also achieving the lowest parameter count and computational cost. In contrast, increasing the number of blocks to two or three did not lead to further improvements in accuracy; rather, performance slightly declined. These findings suggest that in the context of this model and dataset, stacking additional ViT blocks does not provide meaningful benefit and may even introduce redundancy or overfitting.

As a result, a single ViT block was adopted in the final architecture to ensure optimal performance and efficiency.

Table 2.5 The accuracy of different number of ViT block layers.

N	Parameter(k)	FLOPs(M)	Accuracy(%)
1	31.49	13.9	98.46
2	39.53	21.08	97.88
3	47.85	25.88	97.99

This study also investigates how the choice of patch partitioning strategy within the ViT block affects model performance. Four partition sizes were tested: 2×2 , 3×3 , 4×4 , and 5×5 . The experimental results are summarized in Table 2.6.

All partition strategies yielded classification accuracy above 98.2%, with the 3×3

configuration achieving the highest accuracy of 98.46%. Notably, the CNN-based feature extractor outputs feature maps of size 9×9 . Since 9 is divisible by 3, the 3×3 partitioning divides the feature map into 9 non-overlapping patches evenly, without requiring any additional padding. This avoids introducing zero-filled regions and ensures complete spatial coverage, which helps preserve the integrity of local features.

Table 2.6 The test accuracy of the CFV model with various patch size.

P	Parameter(k)	FLOPs(M)	Accuracy(%)
2×2	31.49	13.16	98.25
3×3	31.49	13.9	98.46
4×4	31.49	13.98	98.38
5×5	31.49	13.14	98.28

Considering both model performance and compatibility with the feature map size, the 3×3 patch configuration was selected as the default setting for the ViT block.

The final design aspect investigated was the effect of token dimension d within the ViT block. The token dimension determines the size of the linear embedding vector in ViT-based structures, directly influencing both model expressiveness and the complexity of attention computations. Four values of d were tested: 16, 32, 64, and 96, with corresponding results summarized in Table 2.7.

Table 2.7 The test accuracy of the CFVT model with various token dimension.

d	Parameter(k)	FLOPs(M)	Accuracy(%)
16	24.2	11.98	96.72
32	31.49	13.9	98.46
64	57.51	21.26	98.36
96	100.2	33.36	94.04

When $d=16$, the model achieved the lowest parameter and FLOPs, but also recorded the lowest accuracy of 96.72%. Increasing d to 32 significantly improved accuracy to 98.46% while maintaining a relatively low computational cost, suggesting this setting offers the best trade-off between performance and efficiency. At $d=64$, accuracy remained high, but the model size and FLOPs increased substantially to 57.51K and 21.26M, respectively, making the efficiency gains less justifiable.

Interestingly, when $d=96$, accuracy dropped to 94.04% despite the model being the largest. This indicates that excessively large token dimensions may lead to overfitting or dilute attention focus, ultimately degrading performance.

Based on these results, $d=32$ was selected as the final token dimension for the ViT block, offering an optimal balance between model complexity and classification accuracy.

The total number of parameters in the proposed neural network model is summarized in Table 2.8. The parameters for the convolutional layers are calculated using Equation 2.1, while those for batch normalization (BN) and layer normalization (LN) are determined by Equations 2.2 and 2.3, respectively. The parameters of the linear layers are computed using Equation 2.4. By summing the parameters across all layers, the total number of parameters for the model can be obtained. In this thesis, the CFVT model deployed on the software platform contains approximately 31,000 parameters in total.

Table 2.8 The total parameters of CFVT model.

Operation		Number of parameters	Sum of parameters
Feature Extraction(CNN)	Convolution 1	$3 \times 3 \times 3 \times 8$	216
	Batch	4×8	32

	Normalization 1		
	Convolution 2	$8 \times 3 \times 3 \times 16$	1152
	Batch	4×16	64
	Normalization 2		
	Convolution 3	$16 \times 3 \times 3 \times 24$	3456
	Batch	4×24	96
ViT	Normalization 3		
	Convolution 4	$24 \times 3 \times 3 \times 24$	5184
	Batch	4×24	96
	Normalization 4		
	Convolution 5	$24 \times 1 \times 1 \times 32$	768
	Layer	2×32	64
	Normalization 1		
	Linear 1	32×96	3072
	Linear 2	32×32	1024
	Layer	2×32	64
	Normalization 2		
	Linear 3	32×64	2048
	Linear 4	32×64	2048
	Layer	2×32	64
	Normalization 3		
	Convolution 6	$32 \times 1 \times 1 \times 24$	768
	Batch	4×24	96
	Normalization 5		
	Convolution 7	$48 \times 3 \times 3 \times 24$	10,368

	Batch Normalization 6	4×24	96
Classifier	Convolution 8	24×1×1×24	576
	Batch Normalization 7	4×24	96
	Linear 5	24×2	48
Total parameters:			31,496

$$Conv \text{ parameters} = in_channel \times kernel \text{ size} \times out_channel \quad (2.1)$$

$$BN \text{ parameters} = 4 \times out_channel \quad (2.2)$$

$$LN \text{ parameters} = 2 \times out_channel \quad (2.3)$$

$$Linear \text{ parameters} = feature \text{ map size} \times out_channel \quad (2.4)$$

2.5 Software Result

The confusion matrix is widely used in machine learning and pattern recognition to evaluate the performance of classification models. It illustrates the correspondence between predicted results and actual labels, enabling a detailed examination of the model's performance across different classes and offering insights into its classification accuracy for each category. Table 2.9 presents the confusion matrix of the CFVT model.

Table 2.9 The confusion matrix of CFVT model.

		Predicted		Accuracy
		No collision	Collision	
Accual	No collision	3125	24	99.23%
	Collision	34	592	94.56%

After multiple rounds of experimentation and analysis, the hyperparameters of the proposed CFVT model have been finalized. The complete set of hyperparameters is listed in Table 2.10.

Table 2.10 Hyperparameters setting and loss function.

Hyperparameter	Value
Batch Size	64
Train epoch	100
Learning rate	0.0001
Optimization algorithm	Adam
Loss function	Crossentropy

Chapter 3 Hardware

Implementation

3.1 Trade-offs Between Accuracy and Simplicity in Softmax Alternatives

In the ViT architecture, the Softmax function is essential to the implementation of the self-attention mechanism, as it transforms attention scores into a probability distribution. However, its implementation involves exponential and division operations, which may result in high computational costs on resource-constrained hardware platforms. To address this challenge, two approximation strategies, namely base-2 exponential approximation and Taylor series expansion, were evaluated and empirically compared to identify the most appropriate method.

Replacing the natural exponential e^x in Softmax with a base-2 exponential 2^x can simplify hardware implementation. This is particularly beneficial when dealing with integer inputs, as bit-shifting operations can be leveraged for efficient computation. However, when inputs include fractional values, calculating 2^x requires the use of lookup tables or interpolation techniques, which introduce additional implementation complexity and potential errors.

To evaluate the effectiveness of this approach, a comparison was made between the classification accuracy of the standard Softmax function and its 2^x approximation, as shown in Table 3.1. Under the same ViT architecture and dataset configuration, the standard Softmax model achieved an accuracy of 98.41%, while the base-2 approximation version reached only 98.09%. Although the drop in accuracy is relatively small, it may have a significant impact in applications requiring high precision.

Table 3.1 Comparison of standard Softmax and base-2 exponential approximation in terms of accuracy.

	Standard Softmax	Base-2 Exponential Approx.
Test accuracy	98.41%	98.09%

Taylor series expansion offers a way to approximate the exponential function without requiring lookup tables. Its formulation is shown in Equation 1.15. The impact of different expansion orders on model accuracy was evaluated, and the results are summarized in Table 3.2.

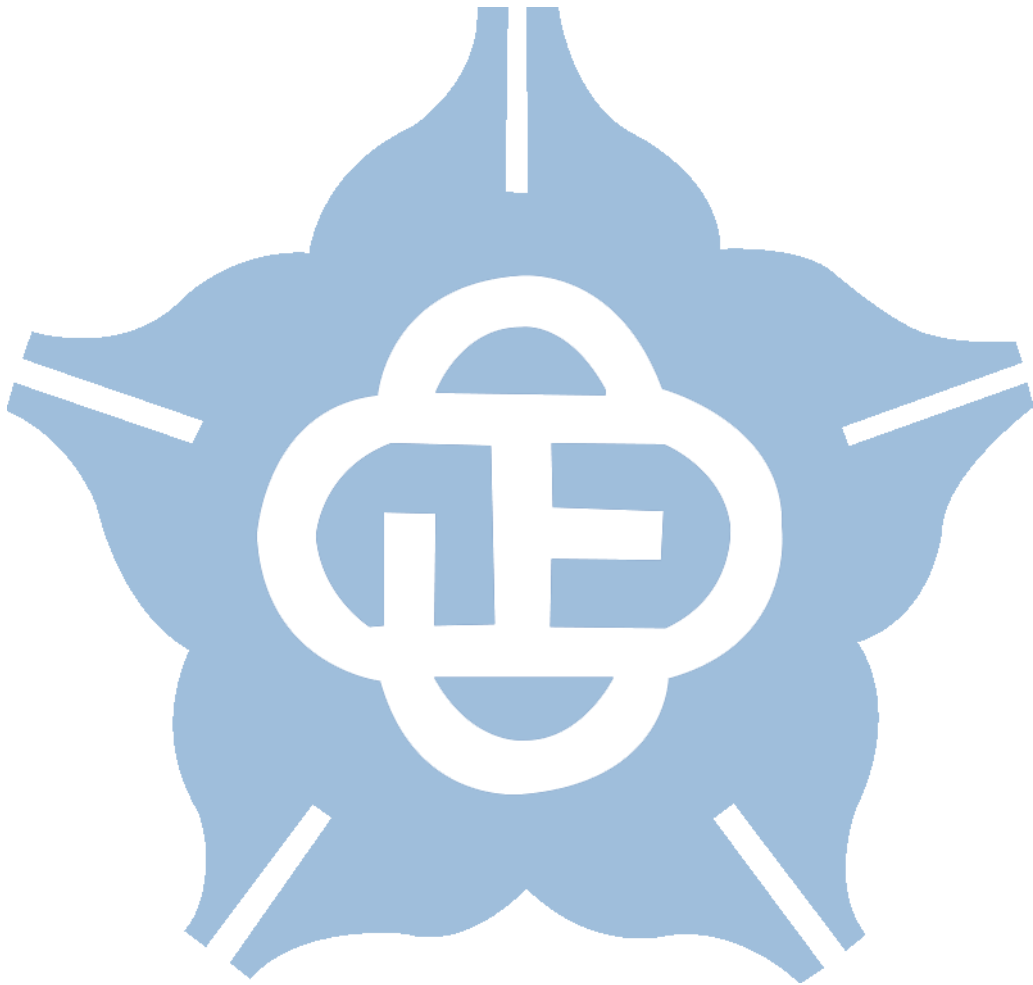
Table 3.2 The test accuracy of the CFVT model with various Taylor polynomial of degree.

Taylor polynomial of degree	Accuracy(%)
1	98.41
2	98.17
3	98.21
4	98.43
5	98.43

According to the experimental results, when $n=4$, the model achieved the highest accuracy of 98.43%, which is even slightly higher than that of the original Softmax function at 98.41%. This demonstrates that with an appropriate expansion order, Taylor series not only preserves model accuracy but also offers the advantages of simple implementation and no need for table lookup.

Based on these findings, this study ultimately adopts the Taylor series expansion as a replacement for the original exponential function $\exp(x)$. After considering the trade-off between computational complexity and accuracy, the first-order expansion

was selected as the final implementation. This configuration achieves an accuracy of 98.41%, which is comparable to that of the original Softmax, while significantly reducing computational effort.



3.2 Fixed-point of Activation

To reduce memory usage and computational cost on hardware-constrained platforms, this study adopts a fixed-point representation for all activation values in the CNNs. Since each convolution layer is followed by a ReLU6 activation function, the output range is bounded between $[0, 6]$, allowing the use of only 3 integer bits to cover the entire effective range.

The number of fractional bits was determined through experimental testing and evaluation. As shown in Table 3.3, different bit-widths for the fractional part were compared, and the results indicate that retaining 5 fractional bits yields sufficient accuracy. Under this configuration, the model achieves its highest accuracy, with quantization error kept within an acceptable range.

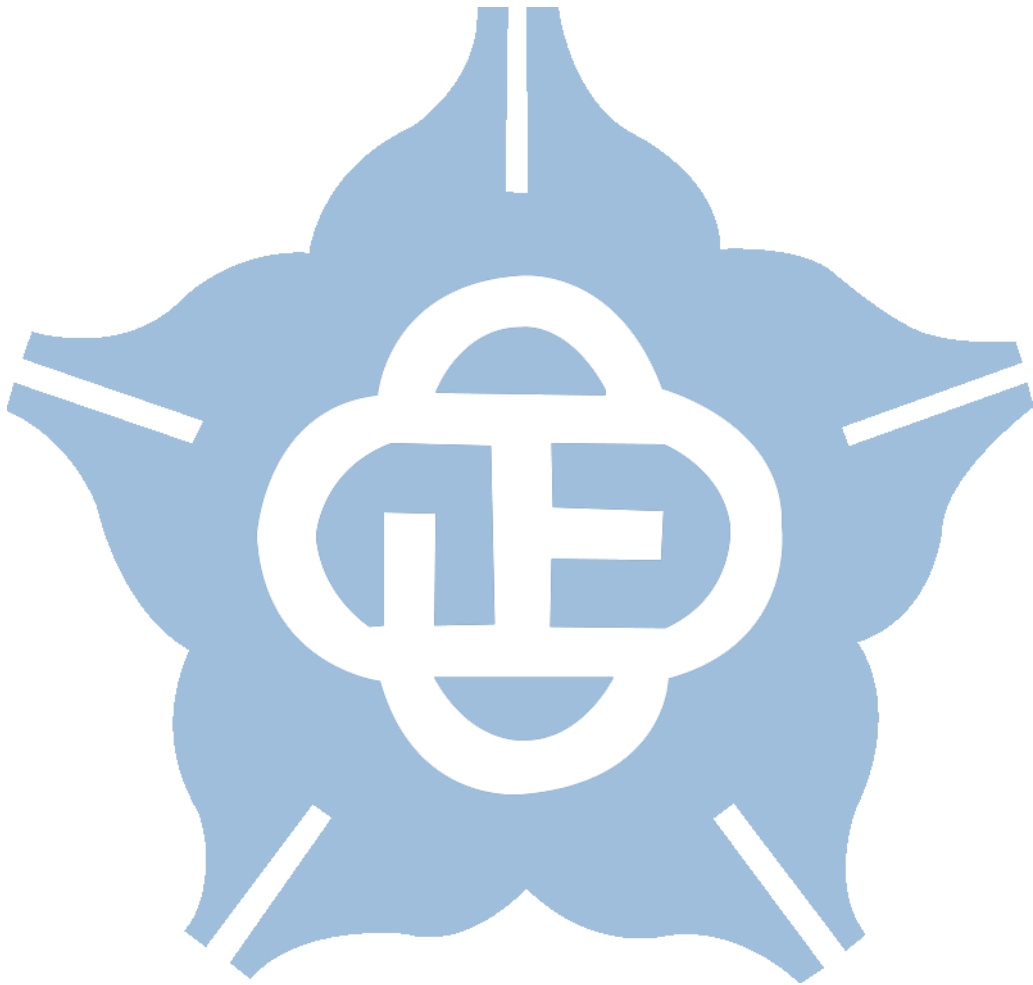
Table 3.3 The accuracy of Activation quantization with different decimal bits.

Integer bits	Decimal bits	Total bits	Accuracy(%)
3	5	8	97.72
3	4	7	96.08
3	3	6	95.66
3	2	5	89.22

Compared to the 32-bit floating-point format, the proposed fixed-point format (3 integer bits + 5 fractional bits) requires only about one-quarter of the original memory. This significantly reduces the storage demand for intermediate activations and parameters. Moreover, fixed-point arithmetic simplifies hardware implementation and can substantially lower power consumption and inference latency, especially on embedded devices lacking floating-point units.

Employing ReLU6 constrains the activation output within a clearly defined range,

thereby enhancing the stability and predictability of the quantization process. This design also facilitates the use of reduced numerical precision without compromising model performance. Taking into account accuracy, memory efficiency, and computational simplicity, a fixed-point format was adopted for activation to enable efficient and practical hardware deployment.



3.3 Fixed-point of Weight

First, fixed-point quantization is applied to the weight parameters of all convolutional and linear layers. By analyzing the distribution of weights after training, it was observed that all weight values fall within the range of $(-1, 1)$. This indicates that only 1 bit is required to represent the integer part, enabling further reduction in storage requirements.

For the fractional part, experiments were conducted to evaluate the impact of different bit-width configurations on model accuracy. As shown in Table 3.4, using 1 integer bit + 8 fractional bits allows for a significant compression of the weight parameters while maintaining accuracy comparable to that of the original floating-point model.

Table 3.4 The accuracy of weight quantization with different decimal bits in fixed point.

Integer bits	Decimal bits	Total bits	Accuracy(%)
1	11	12	98.38
1	10	11	98.30
1	9	10	98.26
1	8	9	98.23
1	7	8	89.54

After completing fixed-point quantization for the model's weights and activation values, the next critical step is to quantize the parameters involved in the Batch Normalization layers. This process not only helps reduce the overall storage requirements of the model but also significantly lowers the computational load during hardware inference, making the model more suitable for deployment on edge devices

or low-power platforms.

During the training phase, each BN layer computes the mean and variance for each mini-batch and uses learnable parameters γ and β to normalize the output features. These four parameters correspond to the standard BN computation formulas, as shown in Equations 1.5 to 1.8. However, applying all four parameters directly during inference increases the number of operations and introduces additional demands for floating-point calculations such as multiplication, division, and square root—operations that are inefficient or costly in a fixed-point environment.

To address this issue, a mathematical transformation is applied to simplify the BN computation. By pre-computing and combining the four original parameters, the BN operation can be reformulated into just two constants as shown in Equations 3.1 and 3.2, representing a new scale and offset. This reformulation allows the entire BN process to be reduced to a simple multiply-and-add operation as shown in Equation 3.3, significantly reducing the complexity and latency of hardware implementation.

$$\gamma' = \frac{1}{\sqrt{\alpha_B^2}} \gamma \quad (3.1)$$

$$\beta' = -(\mu_B \gamma') + \beta \quad (3.2)$$

$$y_i = \gamma' x_i + \beta' \quad (3.3)$$

The simplified parameters also need to be further converted into fixed-point format for inference use. The values of γ' were observed to fall mostly within the range of $[-8, 7]$. Therefore, only 4 bits are required to represent the integer part along with the sign bit.

To determine the appropriate number of fractional bits, a series of experiments were conducted to evaluate the impact of various bit-width configurations on model accuracy. As shown in Table 3.5, when the number of fractional bits was set between

10 and 7, the change in accuracy was negligible. However, once the fractional bit-width dropped below 7, a noticeable decline in accuracy was observed.

Based on this analysis and experimental results, an 11-bit fixed-point format was selected to represent the simplified γ' parameter, consisting of 4 integer bits and 7 fractional bits. This configuration achieves an effective trade-off between hardware efficiency and model accuracy.

Table 3.5 The test accuracy for various decimal bits of BN for γ' .

Integer bits	Decimal bits	Total bits	Accuracy(%)
4	10	14	97.64
4	9	13	97.77
4	8	12	97.67
4	7	11	97.46
4	6	10	86.30
4	5	9	84.22

After completing the fixed-point quantization of γ' , a similar process is applied to β' . Based on observations, the range of β' typically falls within $[-1,1]$, which requires 2 bits to represent both the integer and sign components.

The next step is to determine the bit-width for the fractional part of β' . According to the results shown in Table 3.6, model accuracy remains stable when the number of fractional bits is greater than 4. However, a significant drop in accuracy occurs when the fractional bit-width is reduced to 3 or fewer.

Therefore, in the fixed-point representation of β' , 2 integer bits and 4 fractional bits are used, providing a good balance between model accuracy and hardware efficiency.

Table 3.6 The test accuracy for various decimal bits of BN for β' .

Integer bits	Decimal bits	Total bits	Accuracy(%)
2	15	17	97.48
2	10	12	97.62
2	9	11	97.56
2	8	10	97.67
2	7	9	97.46
2	6	8	97.40
2	5	7	97.51
2	4	6	97.56
2	3	5	79.71
2	2	4	79.83

LN (Layer Normalization) is widely used in Transformer architectures and appears extensively in the ViT model, where LN is applied at the input and output of nearly every module. Unlike Batch Normalization, LN performs normalization across the feature vector of a single sample and does not rely on batch-level statistics. As a result, its behavior remains consistent between training and inference, eliminating the need for additional simplification during deployment.

Each LN layer includes two learnable parameters: the scaling factor γ and the offset β . Based on statistical analysis and experimental validation, it was observed that the values of γ are primarily concentrated within the range of $(-1,1)$. Therefore, a signed fixed-point format using 1 integer bit and 6 fractional bits, totaling 7 bits was adopted . This configuration offers sufficient resolution and dynamic range, and experimental results confirmed that it can maintain model accuracy comparable to the floating-point

baseline, as shown in Table 3.7.

Table 3.7 The test accuracy for various decimal bits of LN for γ .

Integer bits	Decimal bits	Total bits	Accuracy(%)
1	9	10	97.75
1	8	9	97.67
1	7	8	97.51
1	6	7	97.41
1	5	6	96.16
1	4	5	93.43

On the other hand, the absolute values of the offset parameter β are generally very small and have minimal impact on final model predictions. According to empirical data presented in Table 3.8, β remains close to zero during inference in most cases. Consequently, β was quantized as a constant zero. This approach further simplifies hardware design by eliminating a multiply-add operation, resulting in improvements in both memory usage and computational efficiency.

Table 3.8 The test accuracy for various decimal bits of LN for β .

Integer bits	Decimal bits	Total bits	Accuracy(%)
1	8	9	97.51
1	7	8	97.62
1	6	7	97.54
1	5	6	97.59
1	4	5	97.46
1	3	4	97.27

1	2	3	97.43
1	1	2	97.43
1	0	1	97.43
0	0	0	97.43

To enable a more hardware-efficient implementation of LN, this study applies fixed-point representation to its key statistical components—mean and variance. Given that the input feature length is 32, the computation of the mean involves summing 32 elements, while the variance requires squaring and summing those values again. Using floating-point representation would significantly increase hardware resource usage and power consumption. Therefore, converting the mean and variance to fixed-point format simplifies the computation units and reduces the complexity of multiply-accumulate operations. The fixed-point configurations for the mean and variance are presented separately in Table 3.9 and Table 3.10.

Table 3.9 The accuracy of Mean quantization with different decimal bits.

Integer bits	Decimal bits	Total bits	Accuracy(%)
3	9	12	97.43
3	8	11	97.48
3	7	10	97.35
3	6	9	97.40
3	5	8	97.22
3	4	7	97.14
3	3	6	96.11

Table 3.10 The accuracy of Variance quantization with different decimal bits.

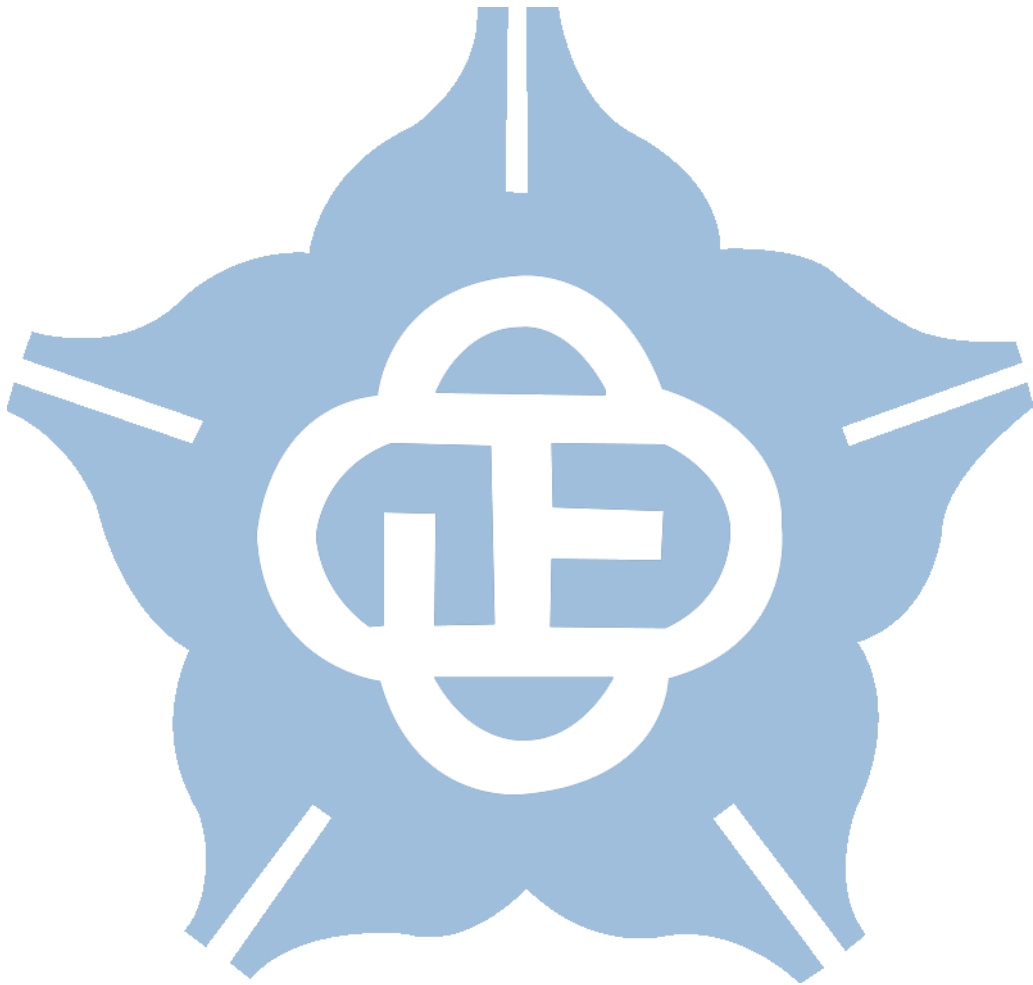
Integer bits	Decimal bits	Total bits	Accuracy(%)
4	13	17	97.44
4	12	16	97.39
4	11	15	97.36
4	10	14	97.41
4	9	13	97.37
4	8	12	97.20
4	7	11	95.74
4	6	10	87.21

Finally, as summarized in Table 3.11, the bit-width requirements for each parameter in the hardware implementation are listed. Compared to traditional models that rely on 32-bit floating-point computation, the proposed method significantly reduces the number of bits needed per parameter. This compression strategy not only lowers overall power consumption but also accelerates inference speed, thereby enhancing the practicality and performance of the model on embedded and low-power devices.

Table 3.11 The selection of fixed-point for hardware implementation.

	Integer bits	Decimal bits	Total bits
Activation	3	5	8
Weight	1	8	9
γ' of batch normalization	4	7	11
β' of batch normalization	2	4	6
γ' of layer normalization	1	6	7

β' of layer normalization	0	0	0
Mean of layer normalization	3	6	9
Variance of layer normalization	4	10	14



3.4 CFVT Hardware Accelerator Architecture

This section will provide a detailed explanation of the hardware implementation and memory usage. Figure 3.1 shows the proposed CFVT block diagram.

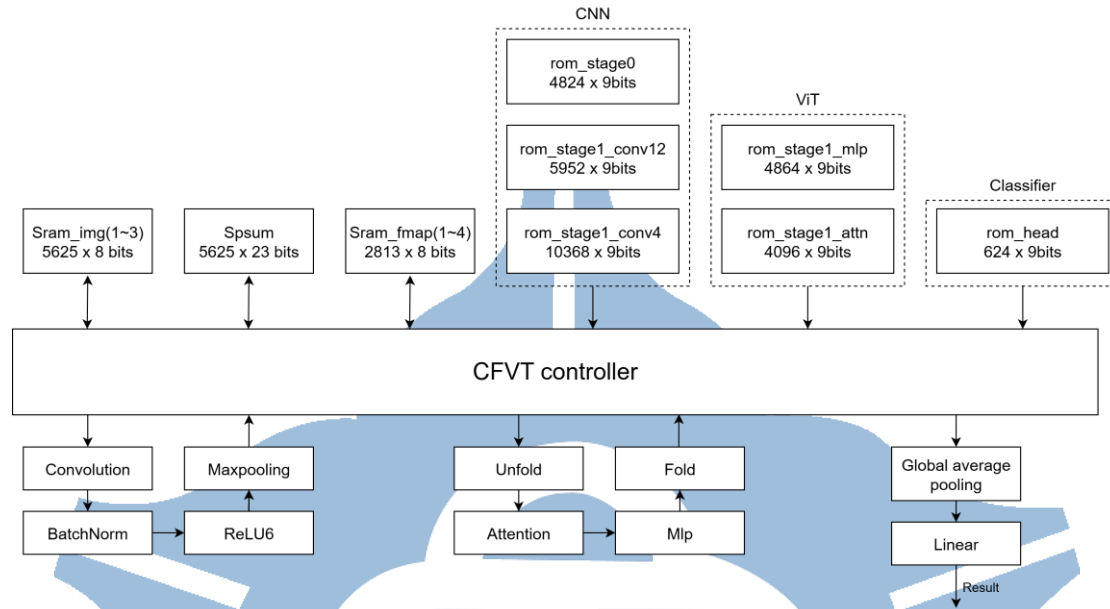


Figure 3.1 The block diagram of proposed CFVT model.

In the overall hardware design, memory requirements primarily come from two types of storage: SRAM and ROM. SRAM is used to temporarily store the input image, intermediate partial sums during computation, and the feature maps generated after each convolutional layer. ROM, on the other hand, stores the weight parameters required by the convolutional and fully connected layers of the model.

The convolution module first applies zero-padding to the input image or feature maps, and then performs convolution operations using the weights stored in ROM. After convolution, the Batch Normalization module uses two pre-simplified parameters to perform a multiply-and-add operation on the convolution outputs. Since these parameters are stored as look-up tables, no additional ROM is required.

The ReLU6 module clips all negative values and limits the output to the range between 0 and 6, providing non-linear activation functionality. The Max Pooling

module performs pooling operations to reduce the spatial resolution of the feature maps, thereby compressing the amount of data to be processed.

During the linear computation stage, the Linear module performs matrix multiplication between the feature vector from the previous layer and the weights stored in ROM. Since these weights are fixed after training, this module only needs to read constant parameters and execute fixed-point multiply-and-add operations, making it both simple and efficient to implement.

Following the linear module, the model proceeds to the LN module, which normalizes the output feature vector to stabilize the distribution and improve convergence and prediction quality. The LN module typically involves two main operations: calculating the mean and variance of the input features for normalization, followed by a linear transformation using quantized scaling and offset parameters

Finally, the Global Average Pooling module computes the average value for each input channel and forwards the result to the final fully connected module for prediction.

Table 3.12 summarizes the memory allocation required by each layer during hardware execution. The input image is a 75×75 RGB color image, with each pixel represented using 8 bits for each of the red, green, and blue channels. These three channels are stored separately in `sram_img1`, `sram_img2`, and `sram_img3`, occupying a total of 135,000 bits.

Once the input data is loaded into SRAM, convolution operations are executed sequentially across layers. The feature maps generated by the first convolution layer are stored in four memory blocks, namely `sram_fmap1` to `sram_fmap4`, with a total memory consumption of 87,616 bits.

Since subsequent convolution layers are followed by Max Pooling operations, the spatial dimensions of the feature maps are halved at each stage. As a result, starting from the third layer, memory requirements for feature maps decrease significantly. To

conserve resources, these smaller feature maps are designed to reuse sram_fmap1 and sram_img1, eliminating the need for additional memory blocks.

Table 3.12 The memory usage for each layer.

Stored data	Feature map size	Total size	Data management
Input image	75×75×3×8	135,000	Sram_img1, Sram_img2, Sram_img3
Conv1	37×37×8×8	87,616	Sram_fmap1, Sram_fmap2, Sram_fmap3, Sram_fmap4
Conv2	18×18×16×8	41,472	Sram_img1
Conv3	9×9×24×8	15,552	Sram_fmap1
Conv4	9×9×24×8	15,552	Sram_img1
Conv5	9×9×32×8	20,736	Sram_img2
Unfold	9×9×32×8	20,736	Sram_img1
LN	9×9×32×8	20,736	Sram_img2
Linear1	3×9×9×32×8	62,208	Sram_fmap2, Sram_fmap3, Sram_fmap4
Get_Weight	9×9×36×8	23,328	Sram_img2
Softmax	9×9×36×8	23,328	Sram_img3
Get_attn	9×9×32×8	20,736	Sram_img2
Linear2	9×9×32×8	20,736	Sram_fmap2
Shortcut	9×9×32×8	20,736	Sram_fmap3
LN	9×9×32×8	20,736	Sram_fmap2
Linear3	9×9×64×8	41,472	Sram_img1
Linear4	9×9×32×8	20,736	Sram_fmap4

Shortcut	9×9×32×8	20,736	Sram_fmap3
LN	9×9×32×8	20,736	Sram_fmap2
Fold	9×9×32×8	20,736	Sram_img1
Conv6	9×9×24×8	15,552	Sram_fmap2
Conv7	9×9×24×8	15,552	Sram_fmap3
Conv8	9×9×24×8	15,552	Sram_fmap2

Through this memory sharing and reuse strategy, the overall SRAM usage of the system is greatly reduced. Additionally, memory modules that are not in use at certain stages are dynamically powered down using power gating techniques to further improve energy efficiency. Detailed explanations of the power management mechanism are provided in Section 3.5.

3.5 Power Gating in Memory Management

Power gating is a commonly used energy-saving technique in modern memory systems. By selectively turning off the power supply to inactive regions of the circuit, it effectively reduces both dynamic and static power consumption. In the context of memory management, the core idea of this technique is to shut down memory blocks that are not currently in use, ensuring that power is supplied only to components actively involved in computation.

The control circuitry plays a crucial role in this process, dynamically determining which memory blocks should be powered on or off based on the system's operational state. This design not only improves overall energy efficiency but also reduces heat generation during chip operation, thereby enhancing system stability and prolonging device lifespan.

The memory power gating mechanism proposed in this study is illustrated in Figure 3.2. The power management module generates control signals pgen1 through pgen10 to manage the power switches for each memory block.

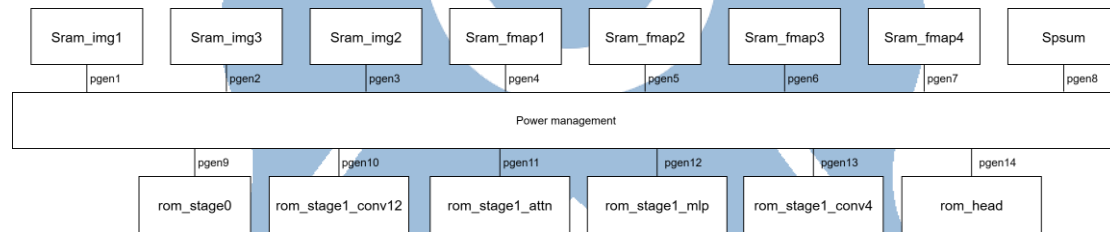


Figure 3.2 The power gating control for memory management.

Detailed information regarding the power management configuration for each memory block within the hardware architecture is provided in Table 3.13 、 Table 3.14.

Table 3.13 The power state in each layer, with a focus on the RAM.

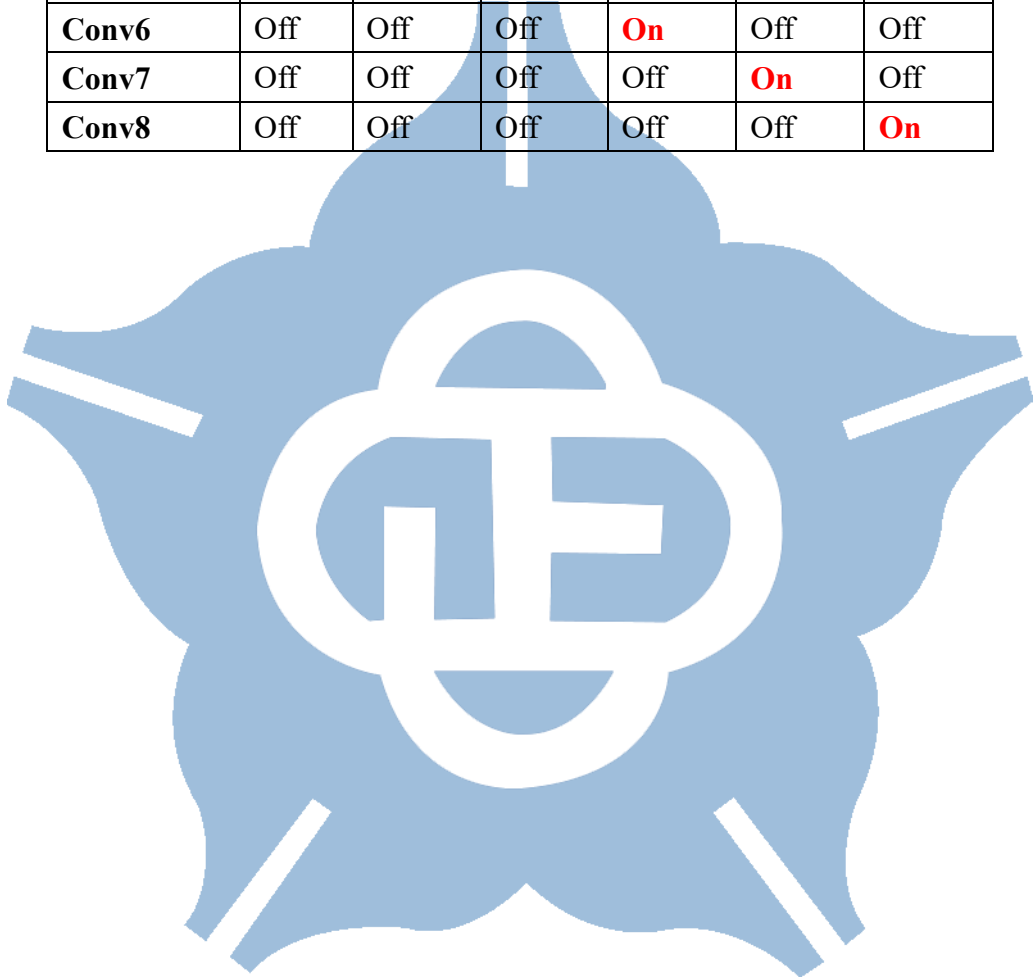
Layer	Power domain							
	Pgen1	Pgen2	Pgen3	Pgen4	Pgen5	Pgen6	Pgen7	Pgen8
Input	On	On	On	Off	Off	Off	Off	On

Conv1	On	On	Off	On	On	On	On	On
Conv2	On	On	Off	On	Off	Off	Off	On
Conv3	On	On	Off	On	Off	Off	Off	On
Conv4	On	On	Off	On	Off	Off	Off	On
Conv5	Off	On	Off	On	Off	Off	Off	On
Unfold	On	On	Off	On	Off	Off	Off	Off
LN	On	On	Off	On	Off	Off	Off	Off
Linear1	On	On	Off	On	On	On	On	Off
Get_Weight	On	On	Off	On	Off	Off	Off	Off
Softmax	On	On	On	On	Off	Off	Off	Off
Get_attn	On	On	Off	On	Off	Off	Off	Off
Linear2	On	Off	Off	On	On	Off	Off	Off
Shortcut	Off	Off	Off	On	On	On	Off	Off
LN	Off	Off	Off	On	On	Off	Off	Off
Linear3	On	Off	Off	On	On	Off	Off	Off
Linear4	Off	Off	Off	On	On	Off	On	Off
Shortcut	Off	Off	Off	On	On	On	Off	Off
LN	Off	Off	Off	On	On	Off	Off	Off
Fold	On	Off	Off	On	On	Off	Off	Off
Conv6	Off	Off	Off	On	On	Off	Off	On
Conv7	Off	Off	Off	On	On	On	Off	On
Conv8	Off	Off	Off	On	On	On	Off	On

Table 3.14 The power state in each layer, with a focus on the ROM.

Layer	Power domain					
	Pgen9	Pgen10	Pgen11	Pgen12	Pgen13	Pgen14
Input	On	Off	Off	Off	Off	Off
Conv1	On	Off	Off	Off	Off	Off
Conv2	On	Off	Off	Off	Off	Off
Conv3	On	Off	Off	Off	Off	Off
Conv4	Off	On	Off	Off	Off	Off
Conv5	Off	On	Off	Off	Off	Off
Unfold	Off	Off	Off	Off	Off	Off
LN	Off	Off	Off	Off	Off	Off
Linear1	Off	Off	On	Off	Off	Off
Get_Weight	Off	Off	On	Off	Off	Off
Softmax	Off	Off	On	Off	Off	Off

Get_attn	Off	Off	On	Off	Off	Off
Linear2	Off	Off	On	Off	Off	Off
Shortcut	Off	Off	Off	Off	Off	Off
LN	Off	Off	Off	Off	Off	Off
Linear3	Off	Off	Off	On	Off	Off
Linear4	Off	Off	Off	On	Off	Off
Shortcut	Off	Off	Off	On	Off	Off
LN	Off	Off	Off	On	Off	Off
Fold	Off	Off	Off	On	Off	Off
Conv6	Off	Off	Off	On	Off	Off
Conv7	Off	Off	Off	Off	On	Off
Conv8	Off	Off	Off	Off	Off	On



3.6 Hardware Implementation Analysis and Comparison

Table 3.15 presents the accuracy results across different stages, from software implementation to hardware realization. It is evident that fixed-point conversion in the CFVT model limit the representable numerical precision of each parameter, leading to a gradual accuracy drop of approximately 1%. Additionally, minor bit-level computation errors observed during the Verilog RTL simulation further contribute to the reduction in accuracy. Nevertheless, the overall model accuracy remains above 97%, indicating acceptable performance despite hardware-induced degradations.

Table 3.15 The test accuracy with different operations.

Operation of each stage	Test accuracy(%)
Build CFVT model	98.46
Softmax Alternatives	98.41
Fixed point	97.41
Verilog Register Transfer Level (RTL)	97.32

Table 3.16 presents the confusion matrix obtained during the Verilog RTL simulation stage for the binary classification task of UAV flight status. The proposed CFVT model demonstrates high accuracy in distinguishing between Normal flight and Collision scenarios. Specifically, 3075 Normal flight samples were correctly classified, with only 74 misclassified as Collision. For the Collision class, 599 samples were correctly identified, while 27 were incorrectly labeled as Normal flight. These results indicate that the fixed-point implementation maintains reliable classification performance, and the replacement of the softmax function with a Taylor series approximation further demonstrates that the model can achieve high accuracy under hardware-friendly constraints.

Table 3.16 Classification confusion matrix at rtl simulation stage.

		True	
		Normal flight	Collision
Predict	Normal flight	3075	27
	Collision	74	599

After multiple adjustments and testing, the proposed CFVT hardware circuit achieves a maximum operating frequency of 100 MHz, with the critical path shown in Figure 3.3. As shown in Equations 3.4, the model prediction time requires a total of 0.041 seconds. The feature extraction stage requires 2,438,999 cycles. The ViT stage takes 1,570,324 cycles. Finally, the classifier stage consumes 103,117 cycles.

```

end
else if(bn_start && bn_cnt == 0) begin
    partial_sum[bn_index] <= (($signed(partial_sum[bn_index]) * gamma) + beta > 0) ?
    ($signed(partial_sum[bn_index]) * gamma) + beta : 0;
end

```

Figure 3.3 Critical path.

$$4,112,440 \times 10 \text{ ns} = 41,124,400 \text{ ns} \quad (3.4)$$

The chip layout of the proposed CFVT hardware design is implemented using TSMC 16nm CMOS technology. As shown in Figure 3.4, it indicates the positions of ROM, SRAM. The chip area is $1.32 \times 1.32 \text{ mm}^2$.

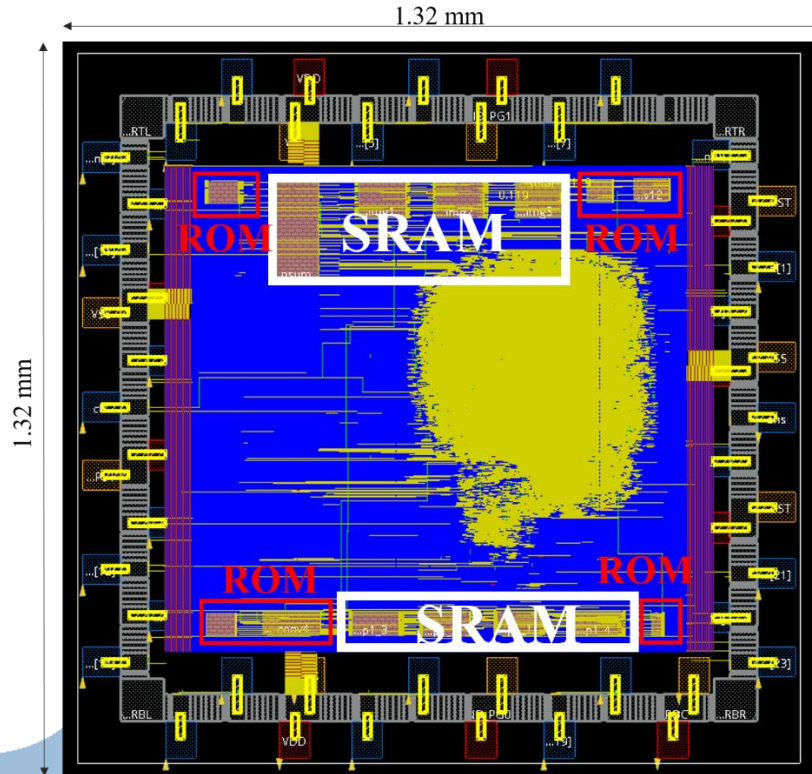


Figure 3.4 The layout of the proposed CFVT circuit.

The implementation results of the proposed CFVT hardware design are shown in Table 3.17. The maximum operating frequency is 100 MHz and a compute throughput of 0.339 giga floating point operations per second (GFLOPS). The proposed CFVT hardware design is implemented in TSMC 16nm CMOS technology.

Table 3.17 The CFVT hardware implementation results.

	The proposed architecture
Technology	16nm FinFET CMOS
Frequency (MHz)	100
Power (mW)	49.2
GFLOPS	0.339
GFLOPS/W	6.891
Chip area (mm^2)	1.74
Parameters (M)	0.031
Accuracy	97.32%

3.7 Automatic Place and Route (APR)

Implementation Process

i. CPODE.L.2/OD.L.5

To address the CPODE.L.2/OD.L.5 DRC violations, we inserted well-tap cells near the reported locations to break the continuous filler chain. This interrupts overly long/continuous active or poly-related geometries and eliminates the violations.

ii. PO.S.22.6

To resolve the PO.S.22.6 DRC violations, the halo width of all memory macros was initially set to 0.96um. After running the Calibre DRC check, the halo width of the memories that still reported PO.S.22.6 violations was adjusted to 0.48um, ensuring compliance with design rules and eliminating the errors.

iii. NanoRoute drc error at IO pad of io_rte

In NanoRoute, set the command shown in Figure 3.5 to prevent io_rte making unnecessary connections to other I/O.

```
dbSet [dbGet -p top.nets.name io_rte ].skipRouting true
```

Figure 3.5 Command.

3.8 Summary

Table 3.18 presents a comprehensive comparison of various approaches applied to the Colanet dataset, covering key aspects such as input image dimensions, quantization strategies, inference time, Parameters, FLOPs, and classification accuracy.

Table 3.18 Comparison with prior methods.

	[8] DoCEIS'2020	[27] Remote Sensing'2020	[28] ECTI- EEC'2024	Proposed work
Dataset	Colanet	Colanet	Colanet	Colanet
Architecture	Vgg16	DCA	MobilenetV3S mall	CFVT
Hardware information	Intel i7 8700 with a Nvidia RTX 2070	NVIDIA Jetson Nano	Raspberry Pi 3B+	16nm ASIC
Input size	224 × 224	224 × 224	224 × 224	75 × 75
Quantization method	N/A	N/A	Post-Training Dynamic Range Quantization	Fixed-point
Parameters(M)	138	2.24	1.11	0.031
FLOPs(M)	1510	602	2.5	13.9
Inference time(s)	N/A	0.182	0.09	0.041

Power(mW)	N/A	N/A	360	49.2
Accuracy(%))	94.55	87.14	77.78	97.32

As shown in Table 3.18, the proposed CFVT model achieves a high accuracy of 97.43% on the Colanet dataset, outperforming recent works such as [8], [27], and [28]. Although the model has a higher FLOPs compared to [28], it is built upon a ViT backbone, which is inherently well-suited for parallel processing. This architectural advantage allows the workload to be distributed across hardware resources, effectively accelerating inference and offsetting the impact of higher computational complexity.

In terms of parameter design, the model leverages a hybrid architecture combining CNN and ViT, retaining the efficiency of convolution for local feature extraction while incorporating ViT's global representation capabilities. This design not only enhances classification performance but also enables the model to achieve better results with fewer parameters.

Overall, the proposed model strikes a strong balance between accuracy and memory efficiency, making it well-suited for low-power and real-time embedded systems, where computational resources are constrained.

Chapter 4 Conclusion and Future works

4.1 Conclusion

This thesis presents an efficient and hardware-friendly model design aimed at UAV collision avoidance. A hybrid architecture combining CNN and ViT is proposed to balance lightweight design with high accuracy. CNNs are employed for efficient local feature extraction, while ViTs provide strong global context modeling. This integration allows the model to achieve high classification performance even with a limited number of parameters, making it suitable for embedded and real-time systems.

To further optimize the computational cost of the ViT structure, the Softmax function is replaced with a hardware-friendly approximation. Instead of traditional exponential and division operations, simplified mathematical methods are introduced to make the implementation more suitable for fixed-point hardware, significantly reducing complexity and improving efficiency.

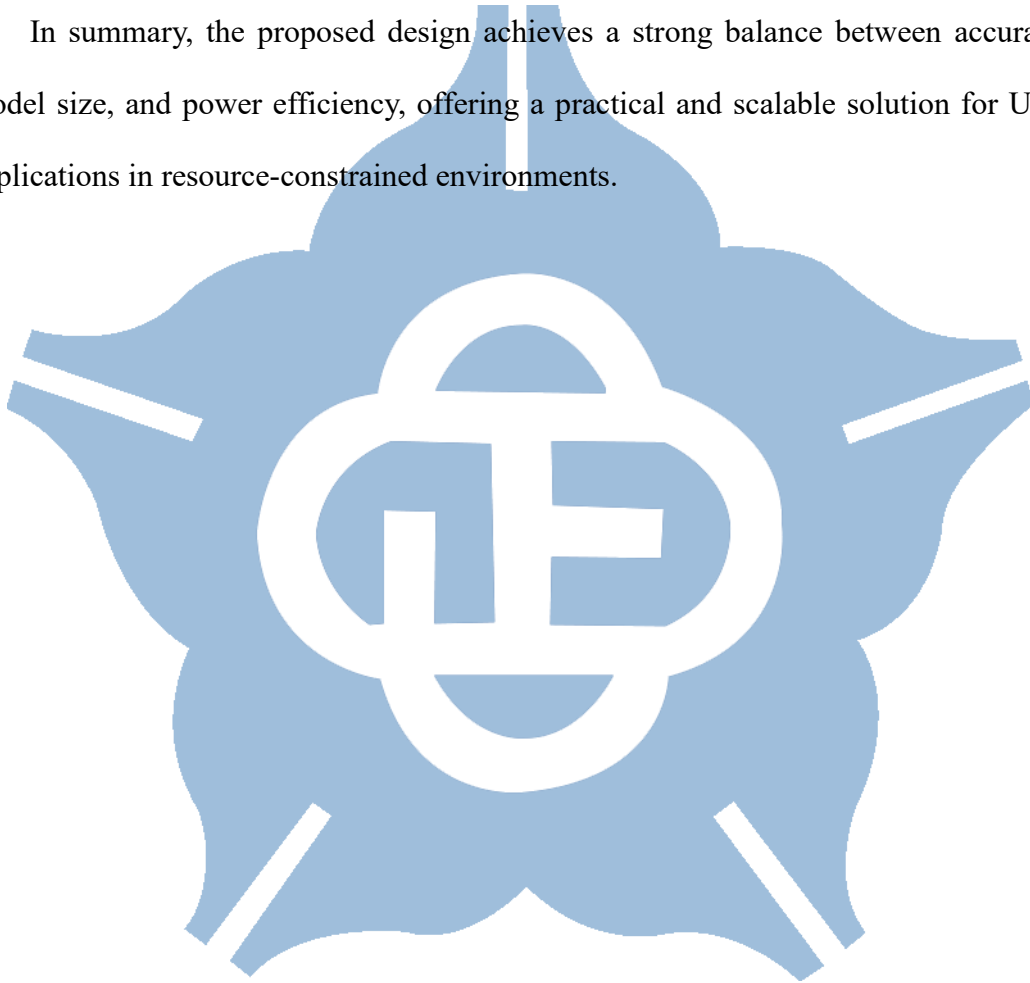
In addition, this work simplifies the batch normalization process by mathematically reducing the four standard parameters to two fixed values, enabling the use of only multiplication and addition operations. This simplification makes it easier to quantize and implement in fixed-point arithmetic, enhancing overall performance during inference.

Throughout the entire model, fixed-point quantization is applied to weights, activations, and normalization parameters to reduce memory usage and computational overhead. Through careful bit-width selection based on empirical evaluation, the model maintains high accuracy while using significantly fewer bits compared to floating-point

representations. This strategy enables deployment on resource-limited hardware platforms, reducing energy consumption and increasing processing speed.

Lastly, power gating is applied in memory management to reduce energy consumption. By selectively turning off unused memory blocks based on operation phases, the system effectively minimizes static power usage without sacrificing performance. This contributes to greater energy efficiency and extended system lifetime.

In summary, the proposed design achieves a strong balance between accuracy, model size, and power efficiency, offering a practical and scalable solution for UAV applications in resource-constrained environments.



4.2 Future works

While the proposed model effectively addresses UAV collision avoidance, there are several promising directions for future enhancement. First, the system could be extended beyond basic obstacle detection by incorporating object recognition. Identifying specific objects would allow the UAV to determine not just the presence of obstacles, but also the optimal direction to navigate around them, enabling more intelligent and context-aware path planning.

Second, although fixed-point quantization was applied to reduce memory and computational costs, future work can explore advanced quantization methods specifically designed for compact Vision Transformer architectures. These methods may help further reduce parameter size and memory usage without sacrificing model accuracy, making the approach even more suitable for deployment on resource-constrained platforms.

Third, low-rank or sparse attention mechanisms could be integrated into the ViT backbone to reduce the computational complexity of the self-attention module. By limiting the number of interactions or compressing the attention matrix, these techniques can significantly improve inference speed and hardware efficiency, while retaining the global representation power of transformers.

Reference

- [1] Hung Pham, Dalil Ichalal, and Said Mammar, “Complete coverage path planning for pests-ridden in precision agriculture using UAV,” in *Proceedings of IEEE International Conference on Networking, Sensing and Control (ICNSC)*, pp. 1-6, Oct. 2020.
- [2] Milan Erdelj, Michał Król, and Enrico Natalizio, “Wireless sensor networks and multi-UAV systems for natural disaster management,” *Computer Networks*, vol. 124, Sep. 2017.
- [3] Khaled Telli, Okba Kraa, Yassine Himeur, Shadi Atalla, and Wathiq Mansoor, “A comprehensive review of recent research trends on unmanned aerial vehicles (uavs),” *Systems*, vol. 11, no. 8, Art no. 400, Aug. 2023.
- [4] Yu-Hsin Hsu and Rung-Hung Gau, “Reinforcement learning-based collision avoidance and optimal trajectory planning in UAV communication networks,” *IEEE Transactions on Mobile Computing*, vol. 21, no. 1, pp. 306-320, June 2020.
- [5] Dong-Wan Yoo, Dae-Yeon Won, and Min-Jea Tahk, “Optical flow based collision avoidance of multi-rotor uavs in urban environments,” *International Journal of Aeronautical and Space Sciences*, vol. 12, no. 3, pp. 252-259, Apr. 2011.
- [6] Shoufeng Chen, Zhihua Yang, Zhentao Liu, Haojie Jin, “An improved artificial potential field based path planning algorithm for unmanned aerial vehicle in dynamic environments,” in *Proceedings of International Conference on Security, Pattern Analysis, and Cybernetics (SPAC)*, Dec. 2017.
- [7] Ali Rida Sahili, Saifeldin Hassan, Saber Muawiyah Sakhrieh, Jinane Mounsef, Noel Maalouf, and Bilal Arain, “A survey of visual SLAM methods,” *IEEE Access*, vol. 11, pp. 139643-139677, Dec. 2023.
- [8] Dário Pedro, André Mora, João Carvalho, Fábio Azevedo, and José Fonseca,

- “Colanet: A uav collision avoidance dataset,” in *Proceedings of Doctoral Conference on Computing, Electrical and Industrial Systems (DoCEIS)*, Jul. 2020.
- [9] Sergey Ioffe and Christian Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of International Conference on Machine Learning (ICML)*, pp. 448-456, Jul. 2015.
- [10] Yann Lecun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- [11] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, May 2017.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep residual learning for image recognition,” in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778, Jun. 2016.
- [13] Karen Simonyan and Andrew Zisserman, “Very deep convolutional networks for large-scale image recognition, arXiv:1409.1556v6 [cs.CV],” *arXiv.org*, Apr. 2015.
- [14] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun, “ShuffleNet: An extremely efficient convolutional neural network for mobile devices,” in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6848-6856, Jun. 2018.
- [15] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size, arXiv:1602.07360v4 [cs.CV],” *arXiv.org*, Nov. 2016.
- [16] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, “MobileNets:

- Efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861v1 [cs.CV],” *arXiv.org*, Apr. 2017.
- [17] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang Chieh Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition(CVPR)*, pp. 4510-4520, Jun. 2018.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Dec. 2017.
- [19] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli, “Scaling neural machine translation, arXiv: 1806.00187v3 [cs.CL],” *arXiv.org*, Sep. 2018.
- [20] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter Liu, “Pegasus: Pre-training with extracted gap-sentences for abstractive summarization,” in *Proceedings of International Conference on Machine Learning (ICML)*, Jul. 2020.
- [21] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V Le, “Xlnet: Generalized autoregressive pretraining for language understanding,” *Advances in Neural Information Processing Systems 32 (NIPS 2019)*, Dec. 2019.
- [22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale, arXiv:2010.11929v2 [cs.CV],” *arXiv.org*, Jun. 2021.
- [23] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou, “Training data-efficient image transformers & distillation through attention,” in *Proceedings of International Conference on*

Machine Learning (ICML), July 2021.

- [24] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of IEEE/CVF International Conference on Computer Vision*, pp. 10012-10022, Oct. 2021.
- [25] Yinpeng Chen, Xiyang Dai, Dongdong Chen, Mengchen Liu, Xiaoyi Dong, Lu Yuan, and Zicheng Liu, “Mobile-former: Bridging mobilenet and transformer,” in *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5270-5279, June 2022.
- [26] Sachin Mehta and Mohammad Rastegari, “MobileViT: light-weight, general purpose, and mobile-friendly vision transformer, arXiv:2110.02178v1 [cs.CV],” *arXiv.org*, Oct. 2021.
- [27] Dário Pedro, João Carvalho, Fábio Azevedo, Ricardo Sacoto-Martins, Luís Bernardo, Luís Campos, José Fonseca, and André Mora, “FFAU—Framework for Fully Autonomous UAVs,” *Remote Sensing*, vol. 12, no. 21, Art no. 3533, Oct. 2020.
- [28] Zufar, Rifqi Nabila and David Banjerdpongchai, “Selection of Lightweight CNN Models with Limited Computing Resources for Drone Collision Prediction,” *ECTI Transactions on Electrical Engineering, Electronics, and Communications*, vol. 22, no.1, Feb. 2024.
- [29] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam, “Searching for MobileNetV3,” in *Proceedings of IEEE/CVF International Conference on Computer Vision*, pp. 1314-1324, Oct. 2019.
- [30] Erwei Wang, James James Davis, Peter Y. K. Cheung, and George Anthony

- Constantinides, “LUTNet: Rethinking inference in FPGA soft logic,” in *Proceedings of IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, Apr. 2019.
- [31] Aishwarya Kagalkar and S. Raghuram, “Cordic based implementation of the softmax activation function,” in *Proceedings of 24th International Symposium on VLSI Design and Test (VDATE)*, Jul. 2020.
- [32] Yuan Zhang, Yonggang Zhang, Lele Peng, Lianghua Quan, Shubin Zheng, Zhonghai Lu, and Hui Chen, “Base-2 softmax function: Suitability for training and efficient hardware implementation,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 9, pp. 3605-3618, Sep. 2022.
- [33] Yue Gao, Weiqiang Liu, and Fabrizio Lombardi, “Design and implementation of an approximate softmax layer for deep neural networks,” in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS)*, Oct. 2020.
- [34] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou, “DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160v3 [cs.NE],” *arXiv.org*, Feb. 2018.
- [35] Qibin Chen, Yijin Teng, Hui Zhang, Kai Jiang, Qiang Duan, Xue Li, “Post-training quantization for longformer with chunkwise quantization granularity and optimized percentile,” in *Proceedings of International Conference on Computer and Communication Systems (ICCCS)*, Apr. 2022.